



Universitatea Politehnica București
Facultatea de Automatică și Calculatoare
Departamentul de Automatică și Ingineria Sistemelor

LUCRARE DE DIPLOMĂ

Reprezentare Eficientă pentru Explicit MPC folosind Rețele Neuronale Artificiale

Absolvent
Raul-Adrian LUNGULESCU

Coordonatori
Assist. Prof. Daniel IOAN
Prof. Florin STOICAN

București, 2026

Cuprins

Listă de figuri	iv
Listă de tabele	v
Listă de algoritmi	v
1 Introducere	1
1.1 Motivație și Context	1
1.2 Formularea Problemei	2
2 Preliminarii	3
2.1 Modelul matematic	3
2.1.1 Modelul neliniar al pendulului invers	3
2.1.2 Modelul liniarizat	4
2.2 <i>Explicit Model Predictive Control (e-MPC)</i>	5
2.2.1 Formularea problemei de optimizare cu constrângeri	5
2.2.2 Soluția mp-QP	5
2.2.3 Legea de control afină (PWA)	6
2.3 Rețele Neuronale Artificiale	7
2.4 Modelul Robust bazat pe MPC	9
2.4.1 Mulțimea invariantă robustă și setul minimal invariant (mRPI)	9
2.4.2 MPC Robust prin intermediul tuburilor de traiectorie	9
3 Învățare Profundă bazată pe Explicit MPC cu Garanții de Robustețe	11
3.1 Calculul mulțimii mRPI utilizând o metodă de aproximare folosind zonotopi	11
3.1.1 Reprezentări zonotopice ale mulțimilor convexe	11
3.1.2 Calculul mulțimii mRPI	11
3.2 Învățare Profundă bazată pe Explicit MPC	12
3.2.1 Date de antrenament	12
3.2.2 Arhitectura Rețelei Neuronale și Procedura de Antrenare	12
Metoda de Pre-antrenare	13
Re-antrenare (Fine-Tuning)	14
3.3 Construcția Tubului Robust Z_{NN}	15
4 Simulare numerică	18
4.1 Rezultatele simulării pe sistemul liniar	18
4.2 Stabilitatea Lyapunov	21
4.3 Validare statistică	22
4.4 Validarea simulării pe sistemul ne-liniar	22
4.5 Analiză comparativă a performanței computaționale	23
5 Validare Experimentală	25
5.1 Configurația sistemului experimental	25
5.1.1 Descriere instalație fizică	25
5.1.2 Implementarea pe microcontroler	25
5.2 Implementarea rețelei neuronale pe microcontroler	26
5.2.1 Extragerea și calcularea legii de control ANN	26

5.2.2	Validarea legii de comandă ANN prin simulare	26
5.3	Validarea legii de comandă în mediul fizic	27
6	Concluzii și Direcții Viitoare	30
6.1	Concluzii	30
6.2	Direcții Viitoare	30
	Bibliografie	32

Listă de figuri

1.1	Ilustrarea principiului Tube MPC [5]	2
2.1	Modelul mecanic al sistemului descris de pendul invers [7]	3
2.2	Vizualizarea unei secțiuni tridimensionale a partiției eMPC pentru sistemul prezentat.	7
3.1	Arhitectura rețelei neuronale.	13
3.2	Comparație între memoriile arhitecturilor alese	13
3.3	Evoluția funcției cost pe parcursul procesului de antrenare.	14
3.4	Evoluția erorii de aproximare în timpul procesului de re-antrenare.	15
3.5	Proiecții 2D ale seturilor mRPI: $\mathcal{Z}$ (albastru) — tub pentru perturbația externă $\mathbf{w}$; $\mathcal{Z}_{NN}$ (mov) — tub extins cu eroarea de aproximare ε ; gri — $A_K \mathcal{Z}_{NN} \oplus \mathcal{D}$. .	17
4.1	Evoluția stărilor — sistem liniar. Benzile de tub $\mathcal{Z}$ (albastru) și $\mathcal{Z}_{NN}$ (mov) înconjoară traiectoriile nominale (linii groase) și reale (linii subțiri).	19
4.2	Eroarea de stare $ x_{eMPC}(\bar{x}_k) - x_{NN}(\bar{x}_k) $ per dimensiune față de tubul robust $\mathcal{Z}_{NN}$, în scală logaritmică.	19
4.3	Comanda aplicată $u(k)$: forță în N.	20
4.4	Eroarea de aproximare online dintre controlerul explicit și rețeaua neuronală, definită ca $e(k) = u_{eMPC}(\bar{x}_k) - u_{NN}(\bar{x}_k) $	20
4.5	Funcția Lyapunov $V(\bar{\mathbf{x}}_k)$ pe scară logaritmică.	21
4.6	Analiză statistică comparativă Monte Carlo ($N_{sim} = 1000$)	22
4.7	Sistem liniar (culori deschise) vs sistem neliniar RK4 (culori închise), cu tuburile liniare $\mathcal{Z}$ și $\mathcal{Z}_{NN}$ suprapuse.	23
5.1	Simularea rețelei $\mathcal{N}(8, 30)$ în neliniar cu perturbație aditivă w_k . Sunt ilustrate cele patru stări ale sistemului, comanda u_k aplicată și banda perturbației aditive.	27
5.2	Răspunsul sistemului <i>pendul invers</i> controlat prin rețeaua neuronală care aproximează regulatorul eMPC.	28
5.3	Evoluția stărilor pe pendul fizic într-un eșantion de simulare	28
5.4	Comanda de control u_{NN} aplicată sistemului pendul-cărucior într-un eșantion de simulare.	29

Listă de tabele

4.1	Benchmark timing și speedup ($N = 100$ stări, $T_s = 20$ ms)	23
4.2	Comparație de memorie: MPC implicit, eMPC explicit și rețea neuronală	24

Listă de algoritmi

3.1	Calcul mRPI prin aproximare zonotopică (bazat pe [10, 12])	12
3.2	Re-antrenare rețea prin fine-tuning ($\epsilon_{\max}$ mic)	15
5.1	Funcția care construiește Legea de comandă ANN	26

1 Introducere

1.1 Motivație și Context

Model Predictive Control (MPC) [1] este una din cele mai utilizate metode de control în industrie, pentru că are capacitatea de a trata explicit constrângerile pe stare și comandă și în același timp de a optimiza o problemă de performanță pe un orizont de timp finit. Pe de altă parte, MPC oferă un interes sporit și în cercetare, datorită ariilor de aplicabilitate în domenii diverse cu scopul de a asigura respectarea cerințelor optime de proiectare.

În realitate MPC implicit prezintă un dezavantaj major ce constă în costul computațional ridicat, în sensul că la fiecare pas este necesară rezolvarea unei probleme de programare pătratică (QP) [2], astfel implementarea în sisteme de timp real este greu realizabilă cu resursele reduse disponibile la acest fel de aplicații în timp real.

O soluție alternativă la MPC implicit este transformarea problemei de optimizare într-o formulare Multi-Parametric Quadratic Programming (mp-QP) [2]. În acest sens, efortul de calcul online este transferat în offline printr-o metodă care pentru întreg spațiul stărilor admisibile înmagazinează soluția ca o lege de control afină pe porțiuni: *piece-wise affine* - PWA [3]. Problema devine una de căutare a regiunilor active, eliminându-se rezolvarea problemei pătratice la fiecare pas.

Metoda descrisă anterior, eMPC, la rândul ei prezintă un dezavantaj semnificativ și anume că în momentul în care numărul de regiuni poliedrale se mărește considerabil din cauza creșterii orizontului de predicție sau al spațiului de stare, memoria necesară și timpul estimat pentru calculul legii de control în regiuni devine neimplementabil pentru sisteme *embedded*.

În ultimă instanță, utilizarea rețelelor neuronale (NN) [3] cu scopul de a aproxima legea de comandă afină este una dintre soluțiile care s-a dovedit utilă în a scădea memoria și timpul de execuție într-un mod eficient pentru implementarea *embedded*, pentru că costul computațional se reduce la înmulțiri matrice-vector. Pentru a scoate în evidență acest aspect se utilizează pentru aproximare o rețea *feedforward* cu activări de tip ReLU care definește prin construcție o funcție afină pe porțiuni [3].

Cel mai important dezavantaj al utilizării rețelelor neuronale în control este dat de faptul că acestea nu conferă nicio asigurare formală a stabilității legii de comandă. În acest sens, utilizarea metodei Robust Tube-MPC [4], care garantează că eroarea de aproximare dintre rețeaua neuronală și comanda eMPC rămâne în interiorul spațiului stărilor și, mai mult, că aceasta converge către origine conform evaluării funcției Lyapunov, permite construirea unor garanții de robustețe pentru sistemul de control.

Scopul lucrării constă în proiectarea unei rețele neuronale care aproximează legea de comandă dată de eMPC, asigurând garanții de robustețe formale față de perturbații aditive mărginite și față de eroarea de aproximare a rețelei. Se utilizează pentru test și simulări pendul invers care are un pronunțat accent de neliniaritate. Lucrarea acoperă următoarele aspecte:

- Determinarea modelului liniarizat pentru pendulul invers și proiectarea algoritmului eMPC pentru calculul regiunilor afine pe porțiuni.

- Calculul mulțimii *minimal Robust Invariant* (mRPI) prin metoda cu aproximare de zonotopi și construcția tubului robust $\mathcal{Z}$.
- Antrenarea rețelei și soluția de reantrenare (fine-tuning) pentru a găsi $\varepsilon_{\max}$ (valoarea cea mai nefavorabilă a erorii de aproximare).
- Validarea prin simulare pe sistemul liniar și neliniar și evaluarea performanței computaționale.

Structura lucrării este următoarea: Capitolul 2 prezintă formulările matematice: modelul matematic al pendulului, formularea eMPC și bazele MPC robust. Capitolul 3 descrie construcția raționamentului propus: calculul mRPI prin zonotopi, arhitectura și procedura de antrenare a rețelei, și construcția tubului $\mathcal{Z}_{NN}$ cu eroarea de aproximare a rețelei. Capitolul 4 prezintă rezultatele numerice comparative, analiza convergenței Lyapunov, validarea pe sistemul neliniar și analiza computațională. În Capitolul 5 se realizează validarea simulării pe instalația fizică, iar Capitolul 6 cuprinde concluziile lucrării și direcțiile viitoare de cercetare.

1.2 Formularea Problemei

Am considerat sistemul în timp discret cu incertitudine aditivă:

$$x_{k+1} = Ax_k + Bu_k + w_k, \quad w_k \in \mathcal{W}, \quad (1.1)$$

unde $x_k \in \mathbb{R}^n$ reprezintă vectorul de stare, $u_k \in \mathbb{R}^m$ este comanda scalară, iar w_k denotă perturbația aditivă mărginită, cu $\mathcal{W} = \{w : \|w\|_{\infty} \leq w_{\text{bound}}\}$.

Se dorește proiectarea unei legi de control $u_k = \varphi(x_k)$ care:

- stabilizează sistemul afectat de perturbații spre origine;
- respectă constrângerile $x_k \in \mathcal{X}$, $u_k \in \mathcal{U}$ pentru orice efect al perturbației $w_k \in \mathcal{W}$;
- poate fi executată în timp real pe sisteme *embedded* cu memorie redusă;
- oferă garanții robuste formale (invarianță, fezabilitate recursivă).

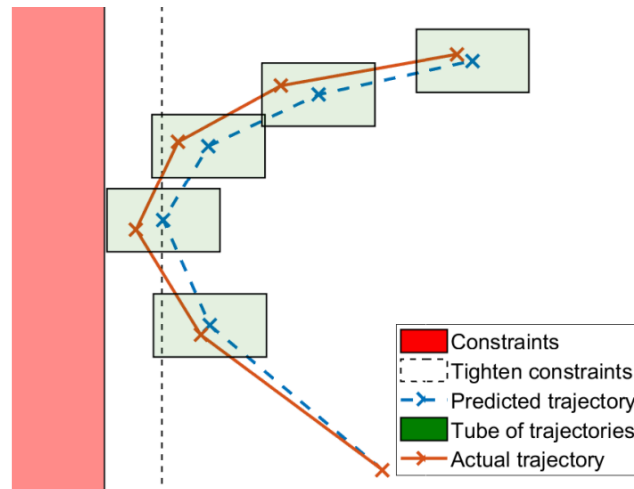


Figura 1.1: Ilustrarea principiului Tube MPC [5]

2 Preliminarii

2.1 Modelul matematic

2.1.1 Modelul neliniar al pendulului invers

Invert pendulum cart [6] reprezintă un exemplu clasic de model dinamic neliniar frecvent utilizat în teoria controlului automat. Acesta este format dintr-un cărucior cu masa m_c care se poate mișca pe orizontală și un pendul invers de masă m_p și lungime l , atașat căruciorului prin intermediul unei articulații fără frecare.

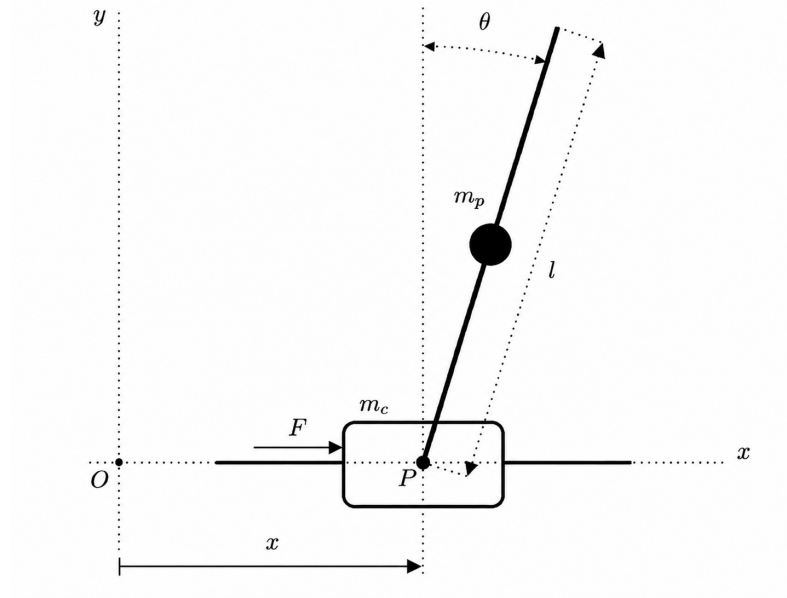


Figura 2.1: Modelul mecanic al sistemului descris de pendul invers [7]

Configurația sistemului poate fi descrisă folosind coordonatele generalizate:

$$q = [x \quad \theta]^T, \quad (2.1)$$

unde x indică poziția orizontală a căruciorului, iar θ reprezintă unghiul pe care pendulul îl formează cu axa verticală.

Dinamica sistemului poate fi derivată folosind relația Euler–Lagrange, bazată pe definiția Lagrangianului:

$$L(q, \dot{q}) = T - V, \quad (2.2)$$

unde T reprezintă energia cinetică totală, iar V energia potențială a sistemului.

Energia cinetică este alcătuită din două componente: energia mișcării de translație a căruciorului și de energia acumulată a mișcării de translație și de rotație (dată de momentul de

inertie I) a masei pendulului:

$$T = \frac{1}{2}m_c\dot{x}^2 + \frac{1}{2}m_p\left(\dot{x}^2 + 2\dot{x}\dot{\theta}l\cos\theta + l^2\dot{\theta}^2\right) + \frac{1}{2}I\dot{\theta}^2. \quad (2.3)$$

Energia potențială a pendulului este dată de relația:

$$V = -mgl\cos\theta. \quad (2.4)$$

Aplicând ecuațiile Euler–Lagrange:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) - \frac{\partial L}{\partial q_i} = Q_i, \quad (2.5)$$

unde Q_i reprezintă forțele generalizate (care conține: $Q_x = F - b\dot{x}$, unde b este constanta de frecare a căruciorului, F – forța externă aplicată căruciorului, și $Q_\theta = -k\dot{\theta}$, unde k este coeficientul de frecare al pendulului), se obțin ecuațiile de mișcare neliniare.

Ecuatiile diferențiale neliniare astfel obținute sunt:

$$(m_c + m_p)\ddot{x} + m_pl\ddot{\theta}\cos\theta - m_pl\dot{\theta}^2\sin\theta = F - b\dot{x}. \quad (2.6)$$

$$\ddot{\theta}(m_pl^2 + I) + m_pl\ddot{x}\cos\theta - m_pgl\sin\theta = -k\dot{\theta}. \quad (2.7)$$

2.1.2 Modelul liniarizat

Pentru a liniariza modelul neliniar în jurul punctului de echilibru $(x, \dot{x}, \theta, \dot{\theta}) = (0, 0, 0, 0)$, se utilizează aproximațiile de unghi mic:

$$\cos\theta \approx 1, \quad \sin\theta \approx \theta, \quad \dot{\theta}^2 \approx 0 \quad (2.8)$$

valabile doar atunci când θ este mic (0° – 20°).

În plus, considerând efectele de frecare neglijabile, $b \approx 0$ și $k \approx 0$ și presupunând intrarea sistemului ca accelerație a căruciorului $u = \ddot{x} = \dot{v} = a$, dinamica sistemului se simplifică semnificativ. De asemenea, pentru că accelerația este luată ca intrare, ecuația liniarizată a poziției căruciorului poate fi omisă.

Rezumând aceste ipoteze, în ecuația liniarizată dată de unghiul pendulului, se obține:

$$\ddot{\theta}(ml^2 + I) + mla - mgl\theta = 0. \quad (2.9)$$

Din această ultimă ecuație se poate deduce reprezentarea în spațiul stărilor. Definind vectorul de stare astfel:

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix}, \quad (2.10)$$

sistemul poate fi scris în forma clasică:

$$\begin{cases} \dot{\mathbf{x}} = A_c\mathbf{x} + B_cu \\ \mathbf{y} = C_c\mathbf{x} + D_cu \end{cases} \quad (2.11)$$

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{mgl}{ml^2+I} & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \\ -\frac{ml}{ml^2+I} \end{bmatrix} a \quad (2.12a)$$

$$\mathbf{y} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix}. \quad (2.12b)$$

2.2 Explicit Model Predictive Control (e-MPC)

2.2.1 Formularea problemei de optimizare cu constrângeri

Se consideră sistemul invariant în timp discret cu dinamica determinată în pasul anterior prin liniarizare:

$$\begin{cases} x_{k+1} = Ax_k + Bu_k \\ y_k = Cx_k \end{cases} \quad (2.13)$$

Pentru a rezolva problema de optimizare MPC se consideră următoarea funcție cost:

$$\begin{aligned} \min_{\mathbf{u}_N} \quad & J = \sum_{k=0}^{N-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_N^\top S x_N \\ \text{s.t.} \quad & x_{k+1} = Ax_k + Bu_k, \quad k = 0, \dots, N-1 \\ & x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}, \quad y_k \in \mathcal{Y} \\ & x_N \in \mathcal{X}_f \end{aligned} \quad (2.14)$$

unde N este orizontul de predicție, $S \succ 0$ este soluția stabilizatoare EMAR (*Ecuția Matriceală Algebrică Riccati* în timp discret) asociată perechii (A, B) cu ponderi $(Q \succeq 0, R \succ 0)$ care garantează că setul terminal $\mathcal{X}_f$ este invariant în buclă închisă.

2.2.2 Soluția mp-QP

Pentru a rezolva problema de optimizare, se definesc vectorii extinși, obținuți prin concatenarea verticală a vectorilor de stare de la momentele $0, \dots, N$, unde N este orizontul de predicție.

$$\mathbf{x} = \begin{bmatrix} x_{1|k} \\ x_{2|k} \\ \vdots \\ x_{N|k} \end{bmatrix}, \quad \mathbf{u} = \begin{bmatrix} u_{0|k} \\ u_{1|k} \\ \vdots \\ u_{N-1|k} \end{bmatrix} \quad (2.15)$$

Folosind proprietatea de liniaritate a sistemului și iterând relația $x_{k+1} = Ax_k + Bu_k$, se obține:

$$\mathbf{x} = Fx_0 + G\mathbf{u}, \quad (2.16)$$

unde F și G sunt construite din (A, B) în forma standard (*lifted*).

Pentru rezolvarea numerică a problemei de optimizare, relația de mai sus $\mathbf{x} = Fx_0 + G\mathbf{u}$ se substituie în funcția cost J pentru exprimarea acesteia exclusiv în funcție de vectorul intrărilor $\mathbf{u}$ și starea inițială x_0 .

Costul devine:

$$J = (Mx_0 + N\mathbf{u})^\top \mathbf{Q}(Mx_0 + N\mathbf{u}) + \mathbf{u}^\top \mathbf{R}\mathbf{u} \quad (2.17)$$

unde $\mathbf{Q}$ și $\mathbf{R}$ sunt matricele bloc-diagonale obținute prin concatenarea matricelor de pondere Q și respectiv R de-a lungul orizontului de predicție.

Prin dezvoltarea și gruparea termenilor în raport cu $\mathbf{u}$, se obține costul:

$$\min_{\mathbf{u}} J = \frac{1}{2} \mathbf{u}^\top \underbrace{2(N^\top \mathbf{Q}N + \mathbf{R})}_{\tilde{Q}} \mathbf{u} + \underbrace{2x_0^\top M^\top \mathbf{Q}N}_{\tilde{H}^\top} \mathbf{u} + x_0^\top M^\top \mathbf{Q}Mx_0 \quad (2.18)$$

cu constrângerile:

$$G\mathbf{u} \leq W + Ex_0 \quad (2.19)$$

Ignorând termenul constant $x_0^\top M^\top \mathbf{Q}Mx_0$, funcția cost se rescrie în forma:

$$\frac{1}{2} \mathbf{u}^\top \tilde{Q}\mathbf{u} + \tilde{H}^\top \mathbf{u}. \quad (2.20)$$

2.2.3 Legea de control afină (PWA)

Aplicând teorema Karush–Kuhn–Tucker [8] pentru funcția cost convexă definită mai sus, se obțin condițiile necesare și suficiente de optimalitate pentru perechea $(\mathbf{u}_N^*, \lambda^*)$:

$$\begin{cases} \tilde{Q}\mathbf{u}_N^* + \tilde{H}^\top x_0 + G^\top \lambda^* = 0 \\ G\mathbf{u}_N^* - Ex_0 \leq W \\ \lambda^* \geq 0 \\ \lambda^*(G\mathbf{u}_N^* - Ex_0 - W) = 0 \end{cases} \quad (2.21)$$

Pentru o mulțime activă fixată $\mathcal{A}$ și mulțimea inactivă $\mathcal{I} = \{1, \dots, m\} \setminus \mathcal{A}$, condițiile KKT se reduc la sistemul liniar:

$$\text{activ: } \begin{bmatrix} \tilde{Q} & G_{\mathcal{A}}^\top \\ G_{\mathcal{A}} & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u}_{\mathcal{A}}^* \\ \lambda_{\mathcal{A}}^* \end{bmatrix} = \begin{bmatrix} -\tilde{H}^\top x_0 \\ W_{\mathcal{A}} + E_{\mathcal{A}}x_0 \end{bmatrix}, \quad \text{inactiv: } \begin{bmatrix} G_{\mathcal{I}} & 0 \\ 0 & -I \end{bmatrix} \begin{bmatrix} \mathbf{u}_{\mathcal{I}}^* \\ \lambda_{\mathcal{I}}^* \end{bmatrix} \leq \begin{bmatrix} E_{\mathcal{I}}x_0 \\ 0 \end{bmatrix}. \quad (2.22)$$

Rezolvând sistemul de egalități se obțin expresiile parametrizate în funcție de starea inițială x_0 pentru comanda optimă și multiplicatorii Lagrange:

$$\mathbf{u}_N^*(x_0) = L_{\mathcal{A}}x_0 + \ell_{\mathcal{A}}, \quad \lambda_{\mathcal{A}}^*(x_0) = S_{\mathcal{A}}x_0 + s_{\mathcal{A}}. \quad (2.23)$$

Această soluție este validă (optimă) pe o regiune a spațiului de stare numită *regiune critică* $\mathcal{CR}_{\mathcal{A}}$, definită prin mulțimea de inegalități:

$$\mathcal{CR}_{\mathcal{A}} = \{x \in \mathbb{R}^n : Z_{\mathcal{A}}x \leq z_{\mathcal{A}}\}, \quad (2.24)$$

unde:

$$Z_{\mathcal{A}} = \begin{bmatrix} A_{\mathcal{I}}L_{\mathcal{A}} - E_{\mathcal{I}} \\ -S_{\mathcal{A}} \end{bmatrix}, \quad z_{\mathcal{A}} = \begin{bmatrix} b_{\mathcal{I}} - A_{\mathcal{I}}\ell_{\mathcal{A}} \\ s_{\mathcal{A}} \end{bmatrix}. \quad (2.25)$$

Ideea centrală a MPC explicit este rezolvarea offline a problemei de programare pătratică (QP) pe întreg spațiul fezabil de stări $\mathcal{X}$. Astfel, în loc să se rezolve online o problemă de optimizare la fiecare pas de eșantionare, legea de control MPC este calculată ca funcție afină de x .

Legea de control $u(x)$ a eMPC este definită ca o funcție afină pe porțiuni continuă peste o partiție a spațiului admisibil $\mathcal{X}_f$ în M regiuni poliedrale:

$$u(x) = \begin{cases} L_{A,1}x + \ell_{A,1} & \text{dacă } Z_{A,1}x \leq z_{A,1} \\ \vdots & \vdots \\ L_{A,M}x + \ell_{A,M} & \text{dacă } Z_{A,M}x \leq z_{A,M} \end{cases} \quad (2.26)$$

Această formulare transformă efortul computațional astfel încât dintr-o optimizare iterativă complexă se construiește o problemă de căutare a legii de control afine pe regiuni poliedrale, implementabilă pe sisteme cu resurse hardware limitate.

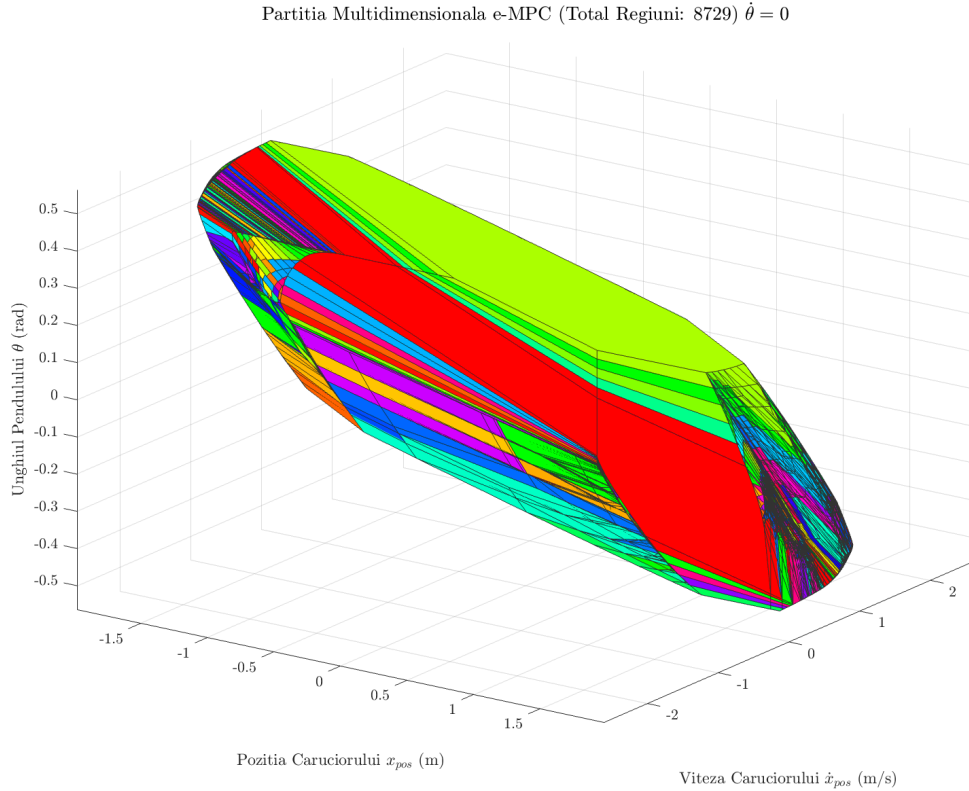


Figura 2.2: Vizualizarea unei secțiuni tridimensionale a partiției eMPC pentru sistemul prezentat.

2.3 Rețele Neuronale Artificiale

O rețea neuronală artificială (ANN) [9] este o funcție parametrică compusă cu straturi succesive de noduri, fiecare strat fiind obținut de o transformare afină (liniară) urmată de o funcție de activare neliniară.

O rețea *feedforward* [9] cu $\ell \in \mathbb{N}^+$ straturi ascunse produce ieșirea $\Phi(x)$ ca o lege de compoziție:

$$\Phi(x) = f^{(\ell+1)} \circ g^{(\ell)} \circ f^{(\ell)} \circ \dots \circ g^{(1)} \circ f^{(1)}(x), \quad (2.27)$$

unde fiecare preactivare $f^{(i)}$ este afină:

$$f^{(i)}(y^{(i-1)}) := W^{(i)} y^{(i-1)} + \mathbf{b}^{(i)}, \quad (2.28)$$

cu matricea de ponderi $W^{(i)}$ și vectorul de bias $\mathbf{b}^{(i)}$. Rețeaua este definită în totalitate de mulțimea parametrilor $\{W^{(1)}, \dots, W^{(\ell+1)}, \mathbf{b}^{(1)}, \dots, \mathbf{b}^{(\ell+1)}\}$ și de alegerea funcțiilor de activare $g^{(i)}$, unde $y^{(i-1)}$ semnifică ieșirea stratului $(i-1)$, cu $y^{(0)} := x$.

Funcția de activare $g^{(i)}$ utilizată este *Rectified Linear Unit* [3] (ReLU), definită astfel:

$$g_j^{(i)}(p^{(i)}) := \max\{p_j^{(i)}, 0\}, \quad (2.29)$$

unde $p^{(i)}$ denotă rezultatul celei de-a i -a preactivări, iar j indică nodul din stratul i .

O proprietate fundamentală [3] a rețelelor ReLU constă în faptul că relația lor intrare–ieșire este *afină pe porțiuni* (PWA). De exemplu, pentru un singur strat ascuns de lățime l_1 , ieșirea rețelei ia forma:

$$\Phi(x) = \mathbf{b}^{(2)} + \sum_{j=1}^{w_1} W_{:,j}^{(2)} \max\{W_{j,:}^{(1)} x + b_j^{(1)}, 0\}, \quad (2.30)$$

unde fiecare neuron fie se activează pe porțiunea liniară, fie se anulează, în funcție de soluția preactivării sale. Hiperplanele $W_{j,:}^{(1)} x + b_j^{(1)} = 0$ definesc frontierele dintre regiunile afine, generând cel mult 2^{l_1} regiuni distincte în spațiul de intrare.

Deoarece legea de control eMPC din (2.26) este ea însăși o funcție PWA cu M regiuni poliedrale, o rețea ReLU reprezintă o soluție naturală pentru reprezentarea acestei legi de comandă. Structura din (2.30) reflectă direct forma PWA, astfel: fiecare neuron activ contribuie cu o regiune liniară, iar compunerea tuturor neuronilor activi reconstituie legea de control locală $L_{\mathcal{A},i} x + \ell_{\mathcal{A},i}$ pe regiunea sa critică $\mathcal{CR}_{\mathcal{A},i}$.

Prin urmare, o rețea $\mathcal{N}(\mathbf{x}; \theta, M, L)$ cu intrarea

$x \in \mathbb{R}^{n_x}$ care se construiește cu funcții de activare ReLU și are $M \geq n_x$ produce o funcție PWA, având limita de regiuni afine pe care le poate genera următoarea relație [9]:

$$\mathcal{N}_{\text{regiuni}} = \left(\prod_{l=1}^{L-1} \left\lfloor \frac{M}{n_x} \right\rfloor^{n_x} \right) \sum_{j=0}^{n_x} \binom{L}{j}, \quad (2.31)$$

unde L este numărul de straturi, M este numărul de neuroni per strat, n_x este dimensiunea spațiului de stare, $\lfloor \cdot \rfloor$ denotă partea întreagă (*floor*), iar $\binom{L}{j} = \frac{L!}{j!(L-j)!}$ reprezintă combinații de L luate câte j .

Relația arată că o rețea neuronală cu L straturi și M neuroni per strat poate genera cel mult $\mathcal{N}_{\text{regiuni}}$ afine distincte, număr care crește exponențial cu adâncimea rețelei L , ceea ce justifică capacitatea rețelelor adânci de a aproxima legi de control de tip PWA cu un număr mare de regiuni, exact ca cele generate de eMPC.

2.4 Modelul Robust bazat pe MPC

2.4.1 Mulțimea invariantă robustă și setul minimal invariant (mRPI)

Fie sistemul liniar discret în $\mathbb{R}^n$, afectat de perturbație mărginită $w(k) \in \mathcal{W}$, unde $\mathcal{W} = \text{ConvHull}\{w_1, \dots, w_{n_w}\}$:

$$x(k+1) = Ax(k) + w(k). \quad (2.32)$$

Definiția 2.1 (Robust Positively Invariant Set). *Un set $\mathcal{X}$ se numește Robust Positively Invariant (RPI) dacă pentru orice $x(0) \in \mathcal{X}$ și pentru orice $w(k) \in \mathcal{W}$, are loc $x(k) \in \mathcal{X}$, $\forall k \geq 0$ [5].*

Definiția 2.2 (Minimal Robust Positively Invariant Set). *Un set Ω_∞ se numește minimal Robust Positively Invariant (mRPI) dacă este un set RPI în $\mathbb{R}^n$ și este conținut în orice alt set RPI al sistemului [5].*

Prin urmare, Ω_∞ (mRPI) poate fi privit ca mulțimea tuturor stărilor în care sistemul poate ajunge, pornind din origine, sub acțiunea perturbațiilor mărginite $w(k) \in \mathcal{W}$, pe un orizont de timp nelimitat.

$$\Omega_\infty = \lim_{k \rightarrow \infty} \bigoplus_{l=0}^k A^l W. \quad (2.33)$$

Importanța formulării de mai sus este subliniată prin invarianța setului, astfel încât odată determinat Ω_∞ , sistemul poate fi stabilizat oricare ar fi perturbațiile adăugate în propagarea dinamicii.

$$\Omega_\infty = A\Omega_\infty \oplus W, \quad (2.34)$$

unde $\oplus$ reprezintă suma Minkowski, definită ca:

$$\mathcal{A} \oplus \mathcal{B} = \{\mathbf{x} = \mathbf{a} + \mathbf{b} : \mathbf{a} \in \mathcal{A}, \mathbf{b} \in \mathcal{B}\}. \quad (2.35)$$

Pe de altă parte, în practică nu se poate implementa o sumă Minkowski infinită, în acest sens se determină o aproximare exterioară de tip ε -mRPI (Z) a mulțimii invariante robuste minimale Ω_∞ [10].

$$Z = \frac{1}{1-\alpha} \bigoplus_{i=0}^{s-1} A_K^i W, \quad (2.36)$$

unde $\alpha \in (0, 1)$ determină gradul de apropiere față de Ω_∞ , iar s este ales astfel încât $A_K^s W \subseteq \alpha W$.

Prin urmare, construind astfel mulțimea Z , aceasta satisface condiția de invarianță:

$$AZ \oplus W \subseteq Z, \quad (2.37)$$

2.4.2 MPC Robust prin intermediul tuburilor de traiectorie

În continuare definim sistemul invariant discret cu incertitudine aditivă:

$$x_{k+1} = Ax_k + Bu_k + w_k. \quad (2.38)$$

De asemenea, introducem sistemul nominal asociat, determinat în lipsa perturbațiilor, de forma:

$$\bar{x}_{k+1} = A\bar{x}_k + B\bar{u}_k. \quad (2.39)$$

Comanda finală aplicată sistemului se calculează prin însumarea componentei nominale (determinată în absența perturbațiilor) și a unui termen de corecție care se leagă de eroarea dintre starea reală și cea nominală:

$$u_k = \bar{u}_k + K(x_k - \bar{x}_k), \quad \text{unde } A_K = A + BK \text{ este Schur.} \quad (2.40)$$

Mulțimea Z prezentată anterior ca aproximare ε -mRPI exterioară și care satisface condiția de invarianță $A_K Z + W \subseteq Z$ este baza pe care se construiesc seturile de constrângere pe stare și comandă:

$$\bar{X} = X \ominus Z, \quad \bar{U} = U \ominus KZ, \quad (2.41)$$

unde $\ominus$ reprezintă diferența Pontryagin, definită ca:

$$\mathcal{A} \ominus \mathcal{B} = \{\mathbf{x} \in \mathcal{A} : \mathbf{x} + \mathbf{y} \in \mathcal{A}, \forall \mathbf{y} \in \mathcal{B}\}, \quad (2.42)$$

Cu toate aceste observații se construiește formularea pentru rezolvarea problemei de optimizare a metodei Robust MPC:

Fie sistemul pe stare, utilizat pentru a descrie comportamentul robust al acestuia în prezența efectelor perturbatoare:

$$x_{k+1} = Ax_k + Bu_k + w_k,$$

$$\text{cu } (A, B) \text{ stabilizabilă; } w_k \in \mathcal{W} = \text{ConvHull}\{w_1, \dots, w_{n_w}\}, \quad u_k \in \mathcal{U}, \quad x_k \in \mathcal{X}.$$

Sistem afectat de perturbație mărginită, $w_k \in \mathcal{W}$.

Comanda optimală se construiește pe baza sistemului nominal astfel:

$$\bar{u}_k^{mpc} = \arg \min_{\bar{x}_k, \bar{u}_k, \dots, \bar{u}_{k+N-1}} \|\bar{x}_{k+N}\|_{2,P}^2 + \sum_{i=0}^{N-1} (\|\bar{x}_{k+i}\|_{2,Q}^2 + \|\bar{u}_{k+i}\|_{2,R}^2)$$

$$\text{subject to } \bar{x}_{k+1} = A\bar{x}_k + B\bar{u}_k$$

$$\bar{u}_{k+i} \in \bar{U}, \quad \bar{x}_{k+i} \in \bar{X}$$

$$\bar{x}_{k+N} \in \bar{X}_N; \quad x_k \in \bar{x}_k \oplus Z$$

Stabilitatea [5]: Legea de control $u_k = \bar{u}_k + K(x_k - \bar{x}_k)$ conferă fezabilitate problemei de optimizare pentru $x_k \in \bar{X}_N \oplus Z$, precum și convergența traiectoriilor sistemului către mRPI (Z), și nu la origine din cauza perturbațiilor aditive mărginite.

3 Învățare Profundă bazată pe Explicit MPC cu Garanții de Robustețe

3.1 Calculul mulțimii mRPI utilizând o metodă de aproximare folosind zonotopi

3.1.1 Reprezentări zonotopice ale mulțimilor convexe

Pentru a construi un MPC robust este necesar construirea un set mRPI, totuși așa cum s-a descris mai sus utilizăm o aproximare exterioară ε -mRPI din cauza spațiilor infinite. Pe de altă parte, aproximarea prin poliedre s-a dovedit a fi extrem de costisitoare din punct de vedere computațional, atunci când numărul de fațete ale poliedrului devine foarte mare. Astfel, am preferat să utilizăm zonotopi, ținând cont că un zonotop [11] este un poliedru, însă cu o reprezentare semnificativ mai compactă.

Un zonotop [11] în $\mathbb{R}^n$ este din punct de vedere geometric suma Minkowski a unui număr mărginit de segmente de dreaptă, caracterizat de centrul $c \in \mathbb{R}^n$ și matricea de generatori $G = [g_1, g_2, \dots, g_p] \in \mathbb{R}^{n \times p}$

$$\mathcal{Z} = \left\{ c + \sum_{i=1}^p \alpha_i g_i \mid \alpha_i \in [-1, 1], i = 1, \dots, p \right\}. \quad (3.1)$$

Zonotopii aproximează foarte bine poliedrele dacă există o mulțime mare de generatori, iar datorită faptului că avem o aproximare exterioară, zonotopul poate include și un poliedru nesimetric. Avantajele zonotopilor sunt remarcabile în calcul, în sensul că suma Minkowski și transformarea liniară a zonotopilor se reduc la simple concatenări de matrici și operații algebrice fundamentale.

3.1.2 Calculul mulțimii mRPI

Mai jos este prezentat algoritmul de aproximare a setului mRPI prin aproximarea cu zonotopi în sensul formulării:

$$Z_s \subseteq Z_\infty \subseteq \frac{1}{1-\alpha} Z_s \quad (3.2)$$

Algoritm 3.1: Calcul mRPI prin aproximare zonotopică (bazat pe [10, 12])

Input: $A_K, \mathbf{w}_{\text{bound}}, \alpha_{\text{target}}$
 1 $s \leftarrow 0, \quad \alpha \leftarrow \infty;$
 2 **while** $\alpha > \alpha_{\text{target}}$ **do**
 3 $s \leftarrow s + 1;$
 4 $\alpha \leftarrow \max_i \frac{(|A_K^s| \mathbf{w}_{\text{bound}})_i}{(\mathbf{w}_{\text{bound}})_i};$
 5 **end**
 6 Generatori zonotop: $G_W \leftarrow \text{diag}(\mathbf{w}_{\text{bound}});$
 7 $G_Z \leftarrow \frac{1}{1-\alpha} [G_W, A_K G_W, \dots, A_K^{s-1} G_W];$
 8 $Z \leftarrow \text{Polyhedron}(G_Z)$ prin funcția de suport;
Output: Z, α, s

Prima secvență a algoritmului urmărește căutarea lui α optim care fixează eroarea de trunchiere a sumei Minkowski până la s pași cu un α_{target} de sub 1%. A doua secvență construiește zonotopul prin propagarea mulțimii de perturbații W în dinamica sistemului, iar prin intermediul acesteia se determină mulțimea invariantă robustă Z .

3.2 Învățare Profundă bazată pe Explicit MPC

3.2.1 Date de antrenament

Datele de antrenament se construiesc pe baza comenzii date de eMPC: $\mathbf{u}^{(i)} = u_{\text{eMPC}}(\bar{\mathbf{x}}^{(i)})$, fiind perechi de forma $\{(\bar{\mathbf{x}}^{(i)}, \mathbf{u}^{(i)})\}_{i=1}^{n_{\text{tr}}}$ ce sunt generate de simularea în buclă închisă a diverselor traiectorii inițiale.

Cu scopul de a acoperi întreg domeniul fezabil al spațiului stărilor se folosesc mai multe strategii de eșantionare a datelor:

Tipul 1 — Uniform global. Eșantionare uniformă în întregul spațiu $\mathcal{X}$:

$$\mathbf{x} \sim \mathcal{U}(\mathbf{x}_{\min}, \mathbf{x}_{\max}).$$

Tipul 2 — Vecinătatea originii. Eșantionare în zona originii, a punctului de echilibru, unde legea eMPC poate varia rapid.

Tipul 3 — Stări extreme. Se alege un $x_j \in [0.8 x_{j,\max}, x_{j,\max}]$, astfel încât să se acopere comportamentul eMPC atunci când starea se îndepărtează de origine, unde liniarizarea începe să nu mai fie corectă.

Tipul 4 — În interiorul lui $\bar{\mathcal{X}}$. Se construiește o mulțime care să aparțină și sistemului nominal $\bar{\mathcal{X}} = \mathcal{X} \ominus Z$ pentru a reduce eroarea de calcul și astfel dimensiunea tubului.

3.2.2 Arhitectura Rețelei Neuronale și Procedura de Antrenare

Rețeaua aleasă pentru a aproxima comanda eMPC este de tip *feedforward*, alcătuită din $L = 8$ straturi ascunse fiecare cu câte $M = 30$ neuroni, propagarea făcându-se prin activare ReLU:

$$\underbrace{4}_{\text{intrare}} \longrightarrow \underbrace{\left[30 \xrightarrow{\text{ReLU}} \right]^{\times 8}}_{8 \text{ straturi ascunse}} \longrightarrow \underbrace{1}_{\text{ieșire}},$$

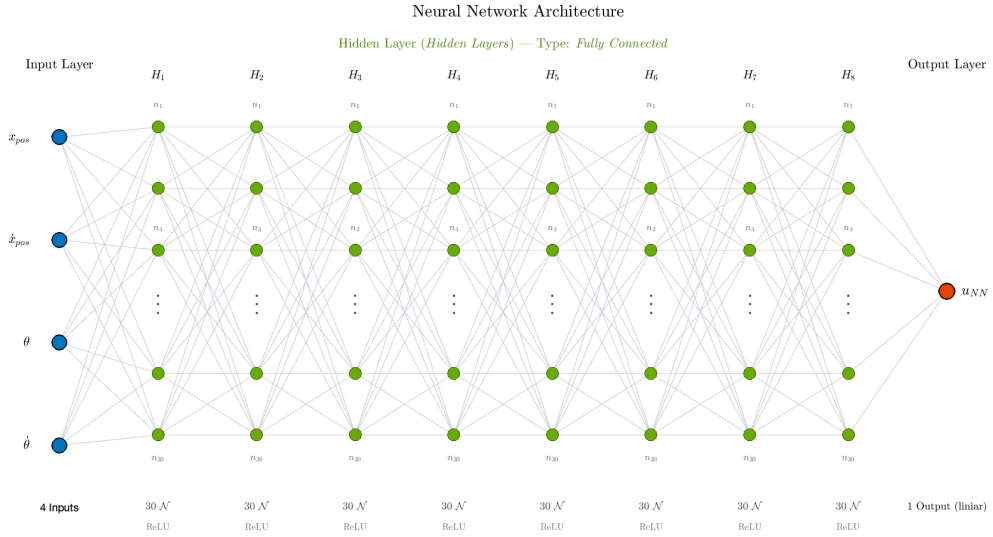


Figura 3.1: Arhitectura rețelei neuronale.

Am ales această rețea $\mathcal{N}(8, 30)$, pentru că o rețea cu mai multe straturi L poate produce mai multe regiuni afine, totuși creșterea lor poate ridica și efortul de calcul. În acest sens, alegerea făcută a constat într-un compromis, astfel încât am constatat că o rețea mai mică de exemplu $\mathcal{N}(6, 12)$ nu a oferit convergența unui ε_{max} la pragul dorit, iar una mai mare ar fi adăugat straturi în plus fără rost, după cum se vede în Figura 3.2.

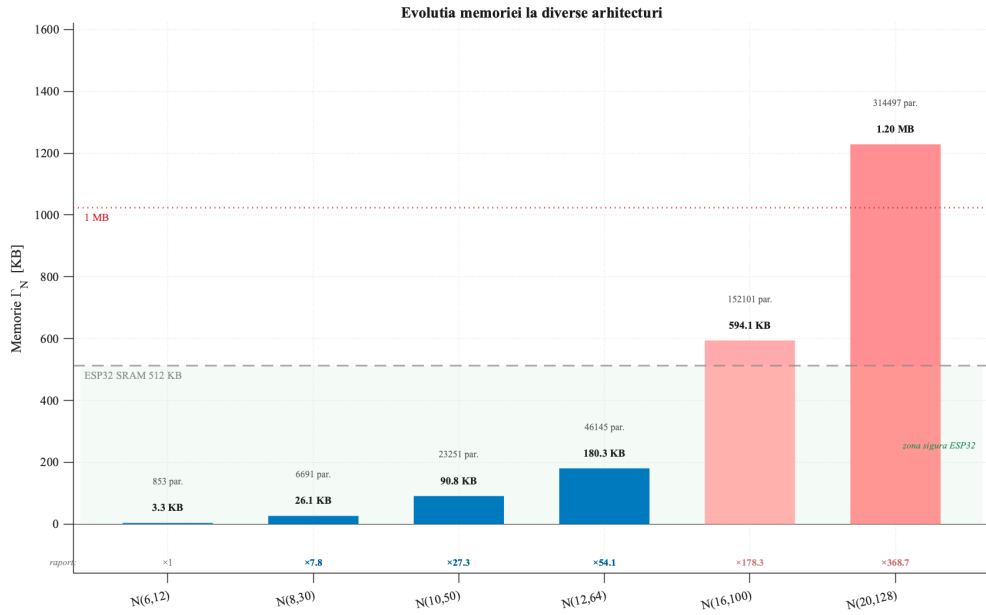


Figura 3.2: Comparație între memoriile arhitecturilor alese

Metoda de Pre-antrenare

Obiectivul este de a minimiza eroarea medie pătratică (MSE - *Mean Square Error*) prin optimizarea parametrilor ce compun rețeaua $\theta = \{W_\ell, \mathbf{b}_\ell\}_{\ell=1}^9$ pe cele n_{tr} date de antrenament:

$$\min_{\theta} \frac{1}{n_{tr}} \sum_{i=1}^{n_{tr}} \|\mathcal{N}(x_{tr,i}; \theta, M, L) - u_{tr,i}\|^2. \quad (3.3)$$

Se utilizează pentru a rezolva problema de optimizare algoritmul ADAM [13], o variantă a gradientului descendent stohastic îmbunătățit. Se folosește o rată de învățare variabilă pe epocile de antrenare (750), astfel să avem inițial o convergență rapidă, și mai apoi o rafinare fină, fără oscilații.

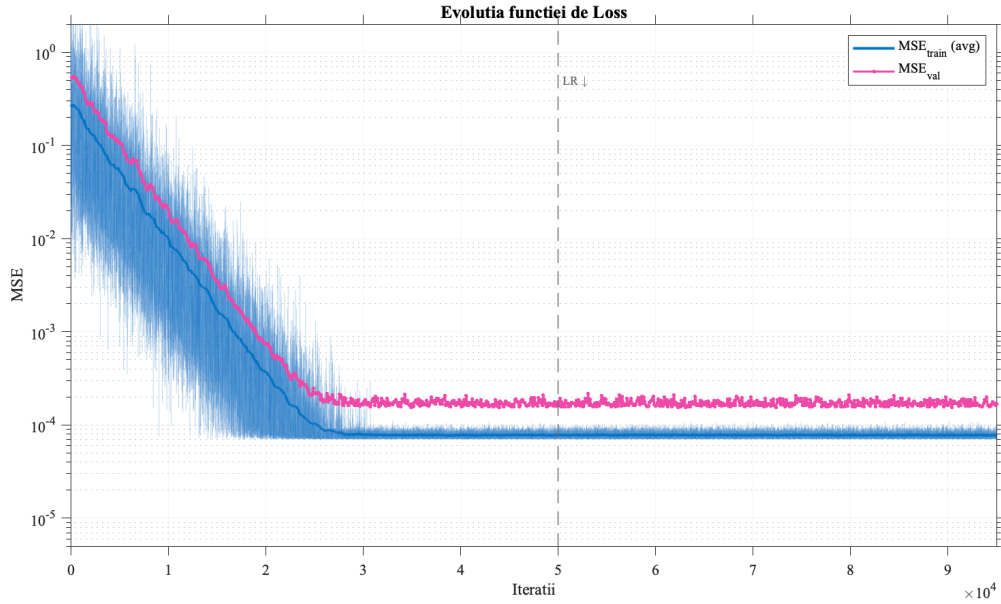


Figura 3.3: Evoluția funcției cost pe parcursul procesului de antrenare.

Graficul din Figura 3.3 prezintă eroarea medie pătratică (MSE) pentru setul de antrenare și pentru cel de validare. Se constată convergența ambelor curbe către valori mici, fără semne de supraînvățare. Linia verticală indică momentul reducerii ratei de învățare pentru a netezi eroarea maximă (*learning rate*).

Re-antrenare (Fine-Tuning)

Pentru a construi tubul Z_{NN} compact pentru cel mai nefavorabil caz minimizăm atât eroarea medie, dar și pe cea maximă:

$$\varepsilon_{\max} = \max_i |u^{(i)} - u_{NN}(\bar{\mathbf{x}}^{(i)})|. \quad (3.4)$$

În acest sens, se introduce o procedură de reantrenare descrisă de Algoritmul 3.2 pentru a sublinia reducerea $\varepsilon_{\max}$ la un prag suficient pentru construcția tubului Z_{NN} compact:

Algoritm 3.2: Re-antrenare rețea prin fine-tuning ($\varepsilon_{\max}$ mic)

Input: $\varepsilon_{\text{target}} = 0.2$, MAX_ROUNDS = 20, rata = 10^{-5}

```

1 for rnd = 1 to MAX_ROUNDS do
2   Calculează  $\varepsilon_k = |u_{\text{eMPC}}(\bar{\mathbf{x}}_k) - \hat{u}(\bar{\mathbf{x}}_k)|, \forall k$ ;
3    $\varepsilon_{\max} \leftarrow \max_k \varepsilon_k$ ;
4   if  $\varepsilon_{\max} < \varepsilon_{\text{target}}$  then
5     break;
6   end
7    $\mathcal{I}_{\text{hard}} \leftarrow \{k \mid \varepsilon_k > \varepsilon_{\text{target}}\}$  - puncte cu  $\varepsilon$  mare
8   Construiește mini-set:  $5 \times \mathcal{I}_{\text{hard}} + 20\%$  din date din preantrenare;
9   Fine-tune rețeaua pe mini-setul construit;
10  Salvează best_net dacă  $\varepsilon_{\max}$  scade;
11 end
12 Restaurează best_net;
```

Algoritmul de minimizare este tot ADAM [13], dar s-au utilizat mai puține epoci (200) pentru o rundă de fine-tuning și o rată de învățare mai mică decât cea inițială pentru a rafina rețeaua în punctele cu $\varepsilon_{\max}$ mare, fără totuși a perturba eroarea medie pătratică în ansamblul ei și a distorsiona rețeaua față de comanda pe care o aproximează. Rezultatele algoritmului de reantrenare au fost surprinse în Figura 3.4, unde se poate vedea comportamentul descris anterior.

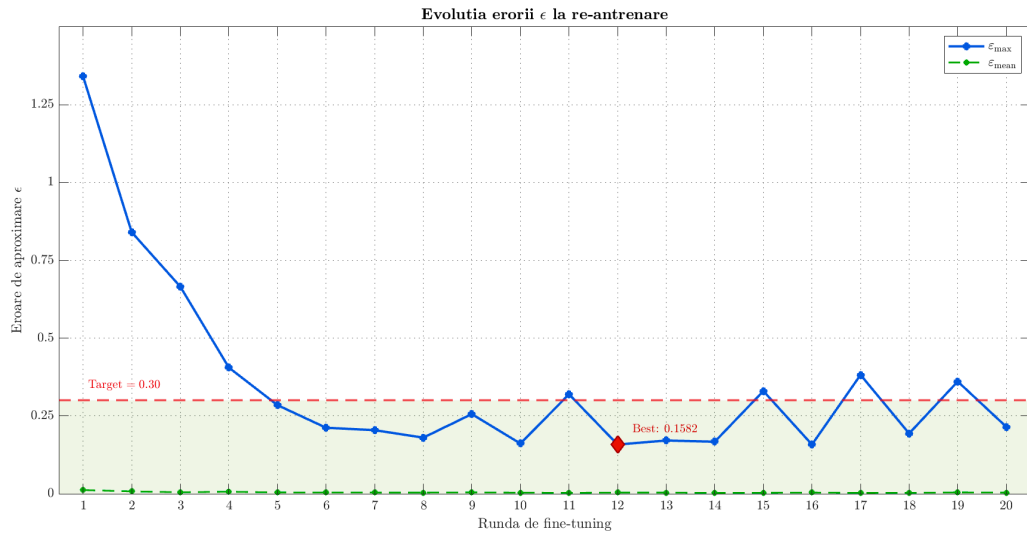


Figura 3.4: Evoluția erorii de aproximare în timpul procesului de re-antrenare.

3.3 Construcția Tubului Robust Z_{NN}

Din cauza faptului că nu avem garanții de stabilitate pentru rețeaua $\mathcal{N}_{8,30}$ obținută anterior și nici măcar certitudinea că aceasta nu părăsește constrângerile pe stare sau comandă, am ales să construiesc tubul robust Z_{NN} care să conțină pe lângă perturbațiile mărginite din setul W și eroarea de aproximare dintre rețea și eMPC în cazul cel mai nefavorabil $\varepsilon_k = u_{\text{eMPC}}(\bar{\mathbf{x}}_k) - u_{\text{NN}}(\bar{\mathbf{x}}_k)$.

Eroarea totală care asigură că tubul nou creat respectă limitările fizice este dată de relația:

$$\mathbf{e}(k+1) = A_K \mathbf{e}(k) + \mathbf{w}_k + B \varepsilon_k = A_K \mathbf{e}(k) + \mathbf{d}_k, \quad (3.5)$$

unde $\mathbf{e}(k)$ este vectorul erorii la momentul k , $A_K = A + BK$ este Schur, $\mathbf{w}_k$ reprezintă perturbația externă mărginită, B este matricea intrărilor, ε_k este eroarea de aproximare, iar $\mathbf{d}_k = \mathbf{w}_k + B\varepsilon_k$ reprezintă perturbația totală pe sistem, aleasă astfel încât să fie mărginită de $\varepsilon_{\max}$ după cum urmează $\mathbf{d}_{\text{bound}} = \mathbf{w}_{\text{bound}} + |B| \varepsilon_{\max}$, unde $\mathbf{w}_{\text{bound}}$ este limita superioară a perturbației externe, iar $\varepsilon_{\max}$ reprezintă eroarea maximă de aproximare aleasă în urma pre-antrenării în cel mai nefavorabil caz de aproximare.

Tubul Z_{NN} este construit cu același Algoritm 3.1 (Rakovic-Zonotop), dar considerând ca perturbație aditivă $\mathbf{d}_{\text{bound}}$.

Pentru a obține o garanție teoretică de stabilitate este necesar construirea tubului robust Z_{NN} în așa fel încât să fie îndeplinită proprietatea de invarianță:

$$A_K Z_{NN} \oplus \mathcal{D} \subseteq Z_{NN}, \quad (3.6)$$

unde $\mathcal{D} = \{\mathbf{d} : |\mathbf{d}| \leq \mathbf{d}_{\text{bound}}\}$.

Definiția 3.1 (Funcție suport [14]). *Fie $\mathcal{S} \subset \mathbb{R}^n$ o mulțime convexă și compactă. Funcția suport a lui $\mathcal{S}$ în direcția $d \in \mathbb{R}^n$ este definită ca:*

$$h_{\mathcal{S}}(d) = \max_{x \in \mathcal{S}} d^\top x. \quad (3.7)$$

Propoziția 3.1 (Funcția suport a unui zonotop [14]). *Fie zonotopul centrat în origine $\mathcal{Z} = \{Gc \mid \|c\|_\infty \leq 1\} \subset \mathbb{R}^n$, cu matricea generatoare $G \in \mathbb{R}^{n \times m}$ și vectorul de coeficienți $c \in \mathbb{R}^m$. Funcția sa suport admite forma explicită:*

$$h_{\mathcal{Z}}(d) = \max_{\|c\|_\infty \leq 1} d^\top Gc = \|G^\top d\|_1 = \sum_{j=1}^m |G_j^\top d|, \quad (3.8)$$

unde am utilizat pentru direcțiile d atât axele canonice $\{\pm e_i\}_{i=1}^n$, dar și coloanele matricei generatoare a zonotopului $\mathcal{Z}_{NN}$, adică $\{\pm G_j\}_{j=1}^m$, pentru o aproximare cât mai bună a tubului în funcția suport caracteristică. Condiția echivalentă pentru incluziune este dată de relația:

$$\left\| G_{A_K Z_{NN}}^\top \mathbf{d} \right\|_1 + \left\| G_D^\top \mathbf{d} \right\|_1 \leq \left\| G_{Z_{NN}}^\top \mathbf{d} \right\|_1. \quad (3.9)$$

Relația a fost testată și verificată pe direcțiile de mai sus, confirmând invarianța aproximării cu zonotopi pentru setul mRPI.

Odată verificată incluziunea și deci invarianța, mai rămâne de văzut dacă tubul construit se încadrează în constrângerile pe stare și comandă astfel:

$$\bar{X} = X \ominus Z_{NN}, \quad \bar{U} = U \ominus K Z_{NN}, \quad (3.10)$$

ceea ce de asemenea este respectat.

Exemplu ilustrativ

Pentru a ilustra construcția celor două seturi mRPI, am considerat sistemul pendulului invers liniarizat, cu parametrii definiți pe larg în Capitolul 2. Utilizând Algoritmul 3.1 –Raković pentru construirea tubului $\mathcal{Z}$ și cu eroarea de aproximare $\varepsilon_{\max} \approx 0.1582$ pentru realizarea

$\mathcal{Z}_{NN}$, am calculat zonotopii mRPI și am realizat proiecțiile lor 2D pe toate perechile de stări $(x_{poz}, \dot{x}_{poz}, \theta, \dot{\theta})$.

În urma simulării se poate observa în Figura 3.5 că $\mathcal{Z} \subset \mathcal{Z}_{NN}$ este respectat în toate cele șase proiecții, confirmând că tubul extins $\mathcal{Z}_{NN}$ înglobează corect tubul nominal $\mathcal{Z}$ și că incluziunea necesară pentru invarianța robustă este satisfăcută, fără ca acesta să depășească restricțiile impuse pe stări: $|x_1| \leq 2.5$ m, $|x_2| \leq 3.0$ m/s, $|x_3| \leq 0.6$ rad, $|x_4| \leq 6.0$ rad/s.

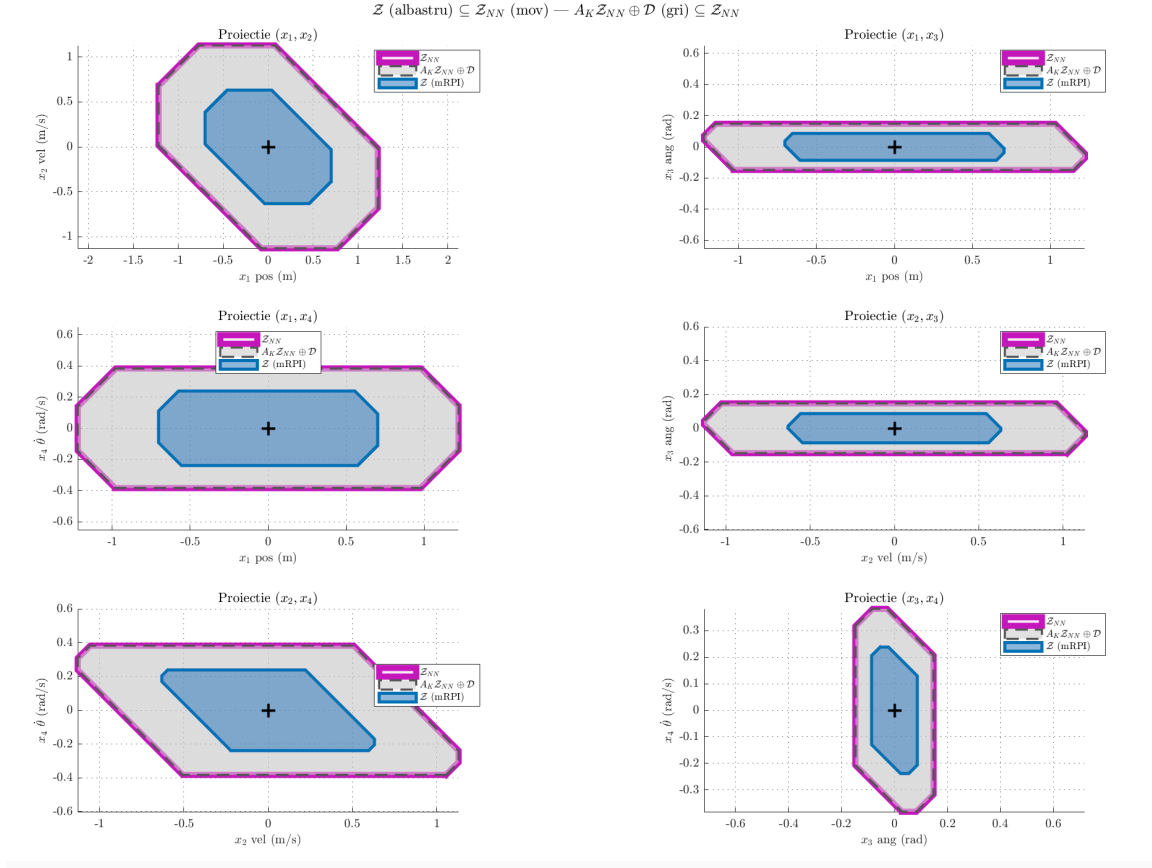


Figura 3.5: Proiecții 2D ale seturilor mRPI: $\mathcal{Z}$ (albastru) — tub pentru perturbația externă $\mathbf{w}$; $\mathcal{Z}_{NN}$ (mov) — tub extins cu eroarea de aproximare ε ; gri — $A_K \mathcal{Z}_{NN} \oplus \mathcal{D}$

4 Simulare numerică

Acest capitol va cuprinde rezultatele comparative între simularea paralelă ale celor trei strategii de control:

- **MPC implicit** [1] (QP online) este algoritmul etalon pe baza căruia se construiesc simulările și comparațiile între metode. Rezolvă o problemă pătratică la fiecare pas online, având cost computațional ridicat.
- **eMPC explicit** [2] (evaluare funcție afină pe porțiuni cu 8729 regiuni) mută efortul în offline prin rezolvarea unei probleme de căutare în regiuni poliedrale. Dacă numărul de regiuni crește exponențial atunci și această căutare prin regiuni necesită un efort computațional mai ridicat.
- **Tube-NN** [9] (aproximare neuronală a eMPC) este soluția care reduce efortul computațional, dar pierde garanțiile de stabilitate formale.

Pentru a confirma ipotezele teoretice din capitolele anterioare, simulările se axează pe analiza comparativă a următoarelor metrici:

- **Comportamentul Dinamic:** Proiectarea traiectoriilor stărilor sistemului pentru cele trei metode de control și a legii de comandă respective atât în dinamica liniară, cât și cea neliniară cu diverse strategii de alegere a stării inițiale.
- **Respectarea Constrângerilor sub efectul perturbațiilor:** Determinarea comportamentului fiecărui controler în a menține stările în interiorul domeniului admisibil $\mathcal{X}$ în prezența incertitudinilor aditive mărginite $w_k \in \mathcal{W}$.
- **Stabilitatea Lyapunov:** Evaluarea în timp a evoluției funcției Lyapunov asociate pentru a confirma convergența traiectoriilor către setul invariant mRPI.
- **Performanța Computațională:** Măsurarea timpului de execuție (analizând atât timpul mediu, cât și cazul cel mai nefavorabil), dar și evaluarea memoriei necesare stocării algoritmilor.

4.1 Rezultatele simulării pe sistemul liniar

Simularea a fost realizată folosind dinamica din Capitolul 2 cu (A, B) obținute în urma discretizării ZoH (*Zero-Order Hold*) a sistemului continuu cu perioada de eșantionare $T_s = 0.02$ s:

$$A = \begin{bmatrix} 1.0000 & 0.0200 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0036 & 0.0200 \\ 0 & 0 & 0.3579 & 1.0036 \end{bmatrix}, \quad B = \begin{bmatrix} 0.0002 \\ 0.0200 \\ -0.0004 \\ -0.0365 \end{bmatrix}. \quad (4.1)$$

Plecând de la starea inițială $\mathbf{x}_0 = [0.5 \text{ m}, 0, 0.15 \text{ rad}, 0]^T$, sistemul fiind afectat de perturbații uniform mărginite $w(k) \in \mathcal{W}$ pe un interval de simulare de $N_{\text{sim}} = 200$ pași (4 s) se construiesc bazele pentru a îndeplini metricile propuse.

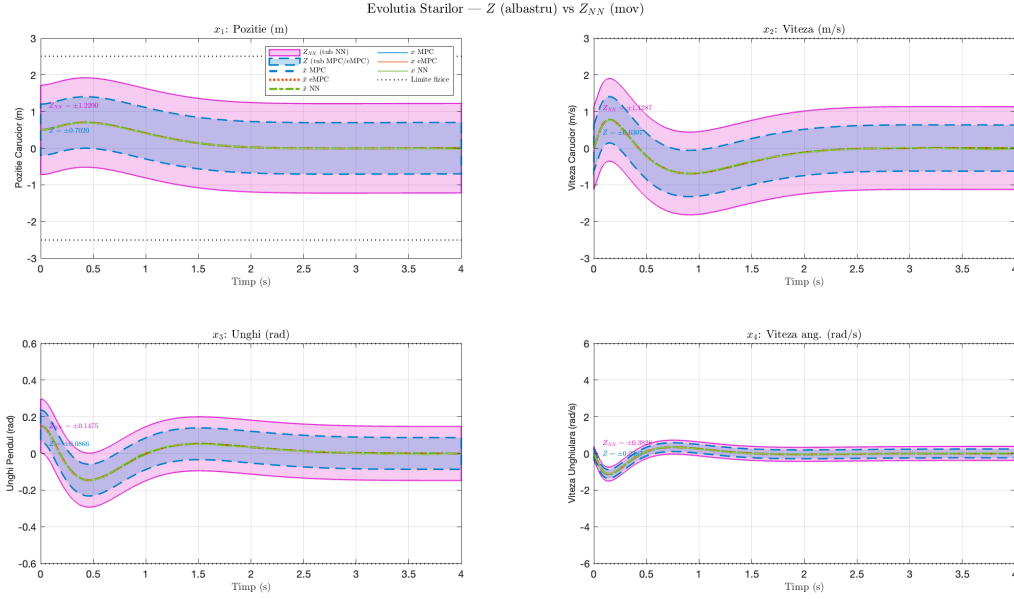


Figura 4.1: Evoluția stărilor — sistem liniar. Benzile de tub Z (albastru) și Z_{NN} (mov) înconjoară traiectoriile nominale (linii groase) și reale (linii subțiri).

Trajectoriile nominale $\bar{\mathbf{x}}$ ale celor trei metode (MPC, eMPC, NN) se conturează ca având o deviație maximă a erorii între metode sub 10^{-3} în toate dimensiunile, asigurând fidelitatea aproximării.

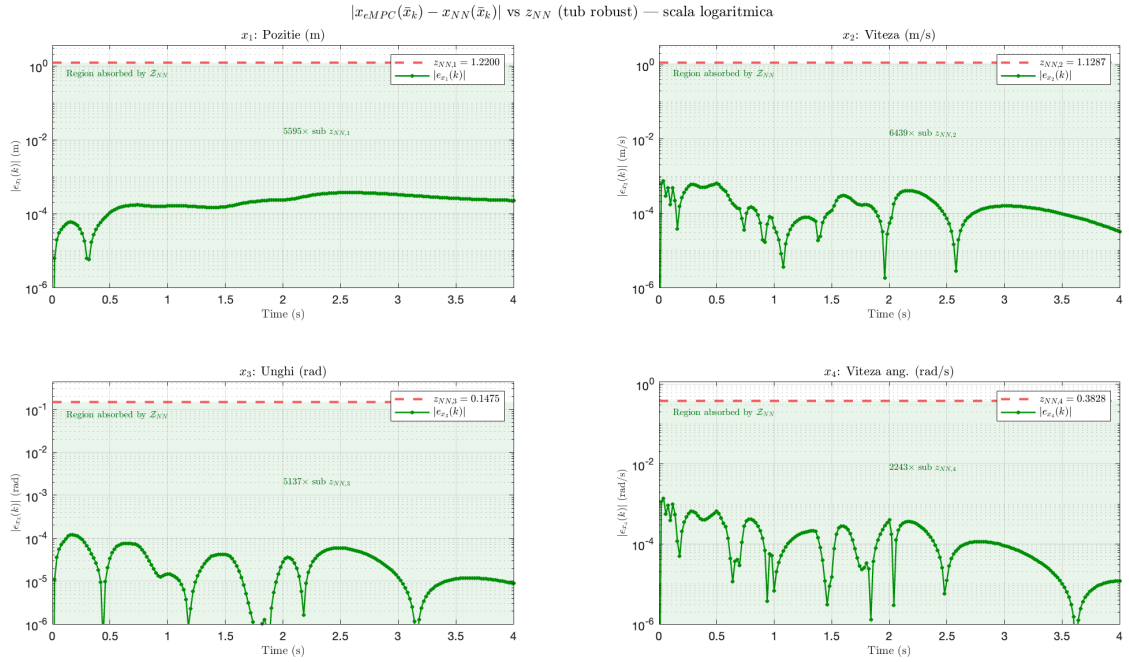


Figura 4.2: Eroarea de stare $|x_{eMPC}(\bar{x}_k) - x_{NN}(\bar{x}_k)|$ per dimensiune față de tubul robust Z_{NN} , în scală logaritmică.

Se observă în Fig 4.1 că toate cele patru traiectorii rămân în ambele tuburi pe întreg intervalul de simulare, respectând condiția de invarianță, astfel sistemul rămâne stabil la perturbațiile externe, respectiv cele interne date de eroarea de aproximare dintre rețea și eMPC.

Starea x_1 (poziție) prezintă un suprareglaj de aproximativ 1.1 m cu timp de stabilizare de 3.5 s, iar x_3 (unghi) atinge un vârf de 0.4 rad cu convergență la 3 s, ambele menținându-se în interiorul constrângerilor fizice.

Tubul Z_{NN} depășește Z cu 60%–79% per dimensiune datorită erorii de aproximare $\varepsilon_{\max} = 0.1582$, însă constrângerile fizice nu sunt încălcate pe nicio dimensiune.

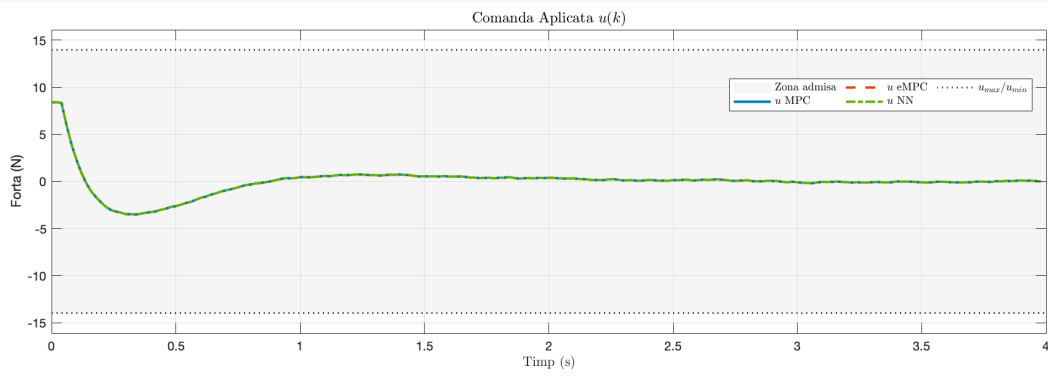


Figura 4.3: Comanda aplicată $u(k)$: forță în N.

După cum se poate observa în Figura 4.3 toate cele trei metode produc comenzi similare, cu un efort maxim la $t = 0$ de ≈ 8.8 N, urmat de scăderea monotonă spre zero. În același timp se observă cum eroarea $e(k) = |u_{eMPC}(\bar{x}_k) - u_{NN}(\bar{x}_k)|$ rămâne la o valoare mică în interiorul tubului Z_{NN} dat de $\varepsilon_{\max}$ pe întreg intervalul simulării, confirmând robustețea tubului.

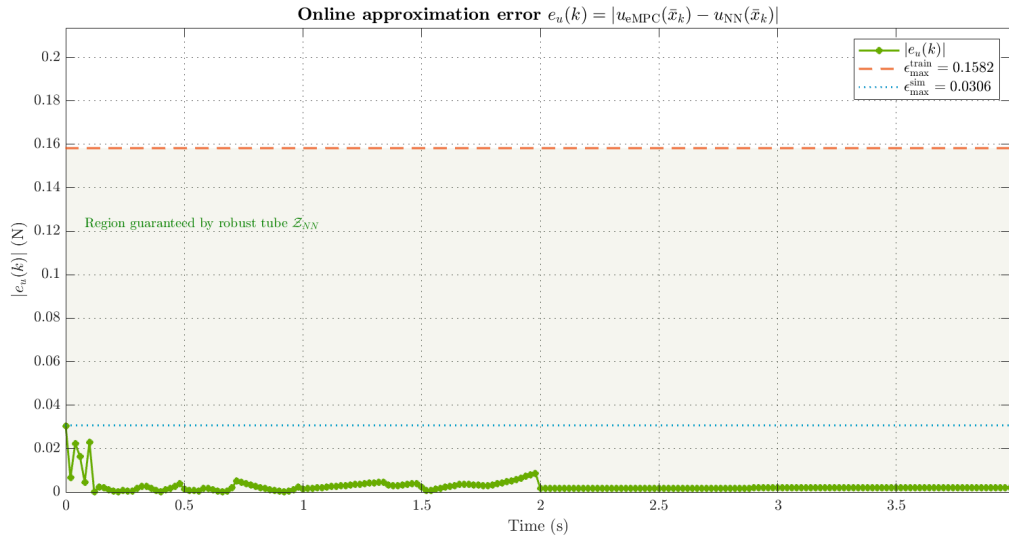


Figura 4.4: Eroarea de aproximare online dintre controlerul explicit și rețeaua neuronală, definită ca $e(k) = |u_{eMPC}(\bar{x}_k) - u_{NN}(\bar{x}_k)|$.

4.2 Stabilitatea Lyapunov

Definiția 4.1 ([5]). *Dacă există $\lambda \in \mathbb{R}_{[0,1]}$ astfel încât $V(x(k+1)) - \lambda V(x(k)) \leq 0$, atunci sistemul $x_{k+1} = Ax_k$ este stabil în sens Lyapunov. Când $\lambda = 1$, funcția $V(x)$ se numește funcție Lyapunov slabă.*

În formularea MPC, matricea P care este soluția stabilizatoare a ecuației DARE (*Discrete Algebraic Riccati Equation*) :

$$A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A + Q \quad (4.2)$$

are două roluri esențiale pentru garantarea stabilității. Pe de o parte, intervine în alcătuirea funcției cost prin adăugarea termenului de penalizare $x_N^T P x_N$ la finalul orizontului de predicție (cost terminal), iar pe de altă parte contribuie la definirea setului terminal invariant $\mathcal{X}_f = \{x \mid x^T P x \leq \alpha\}$ ca constrângere finală prin intermediul căruia se asigură fezabilitatea recursivă.

Funcția Lyapunov $V(\bar{x}_k) = \bar{x}_k^T P \bar{x}_k$, unde matricea P este soluția ecuației algebrice Riccati în timp discret (DARE), măsoară energia sistemului nominal și asigură stabilitatea asimptotică [5], dacă funcția descrește.

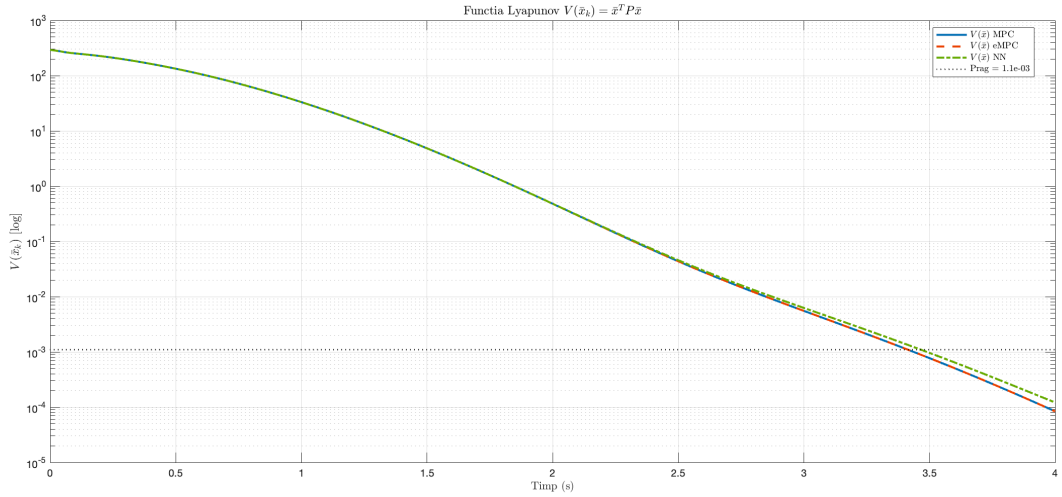


Figura 4.5: Funcția Lyapunov $V(\bar{x}_k)$ pe scară logaritmică.

Toate cele trei curbe descreșc monotonic spre zero, deci spre origine, confirmând că toate metodele converg spre echilibru, ceea ce se observă din faptul că ordinul de mărime cu care scade funcția de la $V_0 \approx 307$ la $V_{200} \approx 1.21 \times 10^{-4}$ tinde spre zero, în cazul rețelei neuronale. O modalitate consistentă de a demonstra convergența funcției în cazul Lyapunov presupune construirea unui set invariant terminal pentru rețeaua neuronală, ceea ce e greu de identificat, din cauza naturii stocastice a algoritmilor de învățare automată.

De asemenea, raza spectrală $\rho(A_K) = 0.9352$ a buclei închise indică un timp de stabilizare $\log(V_{\text{prag}}/V_0)/(2 \log \rho) \approx 3.5$ s pentru sistem în conformitate cu rezultatele din grafice.

4.3 Validare statistică

Descrierea statistică validează rezultatele pe o scară mult mai largă, astfel încât s-au ales 1000 de stări inițiale aleatoare din domeniul fezabil al eMPC care au fost simulate pentru cele două metode eMPC și NN.

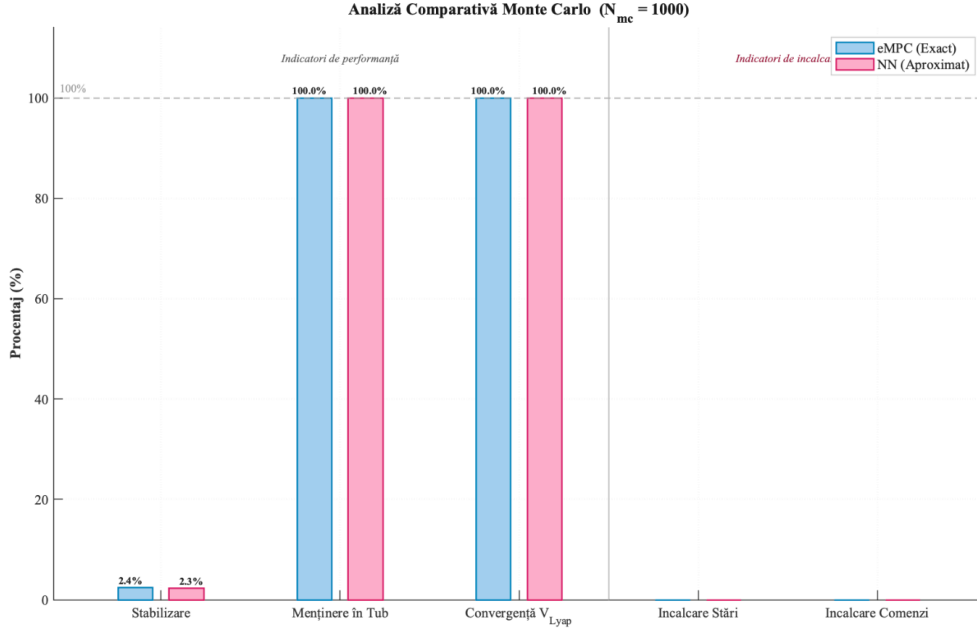


Figura 4.6: Analiză statistică comparativă Monte Carlo ($N_{sim} = 1000$)

În graficul din Fig 4.6 se poate observa cum metricile propuse la începutul Capitolului 4 sunt testate pe o verificare statistică, astfel încât se remarcă următoarele aspecte: toate stările au condus la o dinamică în care eroarea rămâne în tuburi: Z pentru eMPC, respectiv Z_{NN} pentru NN, stările au converș toate spre poziția de echilibru și nu în ultimul rând nu au existat încălcări ale constrângerilor pe stare sau pe comandă, validând astfel capacitatea rețelei de a aproxima comanda dată de controlerul eMPC.

4.4 Validarea simulării pe sistemul ne-liniar

Pentru a valida rezultatele în zona neliniară am considerat sistemul neliniar pe baza ecuațiilor mecanice prezentate în Capitolul 2 și apoi dinamica sistemului a fost determinată prin integrare de tip *Runge-Kutta* de ordinul 4 folosind biblioteca *casadi* [15]. Această metodă presupune aproximarea stării viitoare prin evaluarea sistemului în patru puncte intermediare în modul următor :

$$\begin{aligned}
 k_1 &= f(x_k, u_k), \quad k_2 = f\left(x_k + \frac{\Delta t}{2}k_1, u_k\right), \quad k_3 = f\left(x_k + \frac{\Delta t}{2}k_2, u_k\right), \quad k_4 = f(x_k + \Delta t k_3, u_k), \\
 x_{k+1} &= x_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4).
 \end{aligned} \tag{4.3}$$

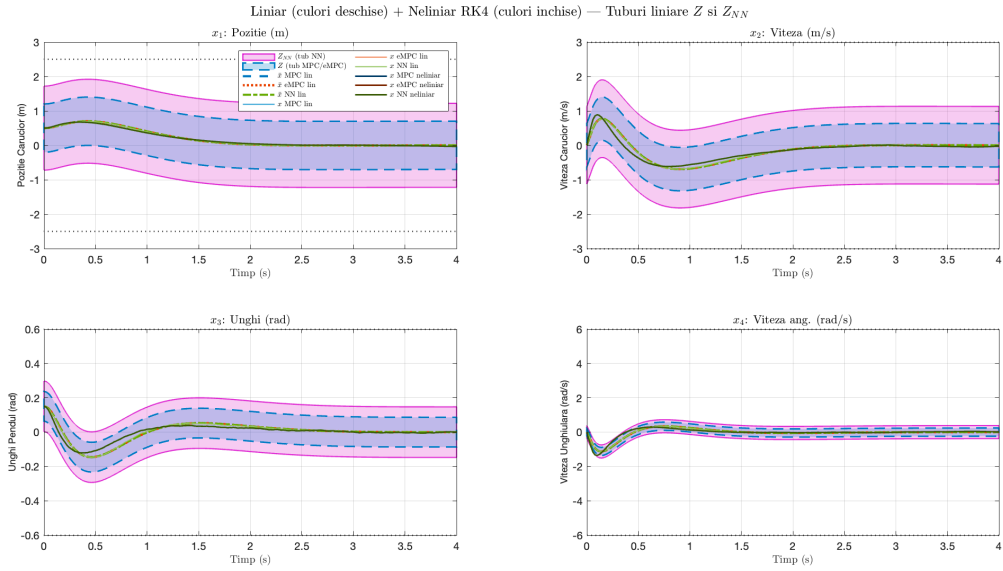


Figura 4.7: Sistem liniar (culori deschise) vs sistem neliniar RK4 (culori închise), cu tuburile liniare $\mathcal{Z}$ și $\mathcal{Z}_{NN}$ suprapuse.

Se observă că pentru stările x_3 (unghi) și x_4 (viteză unghiulară), unde neliniaritatea este pronunțată în ecuațiile ce descriu dinamica, traiectoria sistemului neliniar depășește ușor tubul $\mathcal{Z}$, mai ales în zona în care θ se depărtează de origine, tocmai acolo unde $\sin \theta \approx \theta$ nu mai ține. Acest comportament este o consecință directă a liniarizării în jurul punctului de echilibru: tubul $\mathcal{Z}$ garantând invarianța exclusiv pentru modelul liniar. Pe de altă parte, traiectoriile neliniare părăsesc tubul $\mathcal{Z}_{NN}$ pentru x_4 (viteză unghiulară), demonstrând că tubul nu tinde să fie conservativ, păstrând performanțele de robustețe. Totodată, din cauza liniarizării, erorile date de neliniaritate sunt mai mari decât erorile de aproximare a rețelei $\varepsilon_{\max} = 0.1582$, putând fi construite metode mai avansate precum tuburi neliniare de robustețe.

4.5 Analiză comparativă a performanței computaționale

Pentru a măsura timpul de evaluare a fiecărei metode (timpul necesar calculării legii de control $u(x)$) s-a utilizat $N_{\text{bench}} = 100$ stări aleatoare din setul de date de antrenament cu ajutorul funcției `timeit` din MATLAB. Pentru a elimina overhead-ul de JIT s-au folosit 10 apeluri premergătoare (warm-up).

Tabelul 4.1 descrie timpii obținuți în *ms* pentru metodele clasice de MPC împreună cu rețeaua neuronală. S-au calculat timpul mediu, timpul maxim de execuție (WCET) și deviația standard pentru metodele enunțate anterior.

Tabelul 4.1: Benchmark timing și speedup ($N = 100$ stări, $T_s = 20$ ms)

Metoda	Medie (ms)	WCET (ms)	Std (ms)	Speedup avg	Speedup WCET
MPC implicit	3.8296	15.1258	1.3098	1.0×	1.0×
eMPC explicit	0.1449	1.5134	0.3304	26.4×	10.0×
NN algebric	0.0048	0.0449	0.0041	795.7×	336.9×

Rezultatele în timp confirmă utilizarea metodei (rețele neuronale) în aplicații de timp real.

O altă parte esențială a performanței pentru sisteme *embedded* o reprezintă memoria ocupată. Se poate observa că și la acest capitol aproximarea prin rețele neuronale e net superioară.

Tabelul 4.2: Comparație de memorie: MPC implicit, eMPC explicit și rețea neuronală

Metoda	Tipul memoriei	Memorie (KB)	Sursa memoriei	Observație
MPC implicit	Matrice QP online	2.93	H, A_{eq}, A_{ineq}	Solver QP online
eMPC explicit	Regiuni poliedrice	3 347.2 (≈ 3.27 MB)	8729 regiuni	64× față de NN
NN algebric	Ponderi rețea	52.27	6691 parametri	Referință

Metoda eMPC nu poate fi implementată în practică pe aplicații *embedded*, din cauza memoriei excesiv de mare dată de numărul mare de regiuni generate de setul de constrângeri strânse, de alegerea ca set terminal a spațiului invariant maxim, și nu de orizontul de predicție $N = 5$. MPC pe de altă parte presupune rezolvarea unei probleme QP la fiecare pas în mod online, în timp ce NN face doar înmulțiri de matrici o singură dată.

Memoriile relevante pentru sisteme în timp real sunt date de relațiile [9]:

- Memoria pentru MPC explicit:

$$\Gamma_K = \alpha_{\text{bit}} [n_h(n_x + 1) + n_f(n_x n_u + n_u)], \quad (4.4)$$

unde n_h — nr. hiperplane, n_f — nr. funcții afine, n_x — nr. stări, n_u — nr. intrări.

- Memoria pentru rețeaua neuronală:

$$\Gamma_{\mathcal{N}} = \alpha_{\text{bit}} [(n_x + 1)M + (L - 1)(M + 1)M + (M + 1)n_u], \quad (4.5)$$

unde α_{bit} — memoria unui număr real, L — nr. straturi ascunse, M — nr. neuroni per strat.

5 Validare Experimentală

5.1 Configurația sistemului experimental

5.1.1 Descriere instalație fizică

Pentru a întări rezultatele descrise în Capitolul 4 unde au fost realizate simulările numerice se consideră sistemul fizic cu pendul invers [7] utilizat pentru validarea experimentală a metodei propuse de control ANN.

Sistemul fizic [7] este construit dintr-un mecanism cărucior-pendul care are capacitatea să descrie mișcarea de translație a mobilului, respectiv de rotație a pendulului. Căruciorul se deplasează pe o șină cu ghidare liniară, acționat de un motor pas cu pas NEMA 17 prin intermediul unei curele dințate, motorul este comandat de driverul TMC2209 prin semnale STEP/DIR. Viteza căruciorului este dată de frecvența impulsurilor generate pe pinul STEP, astfel încât fiecare impuls corespunde unui pas al motorului.

Pentru a determina în timp real starea sistemului $x_k = [x, \dot{x}, \theta, \dot{\theta}]^\top$ se folosesc doi senzori (encoderi optici): un encoder de poziție, plasat pe șină, care măsoară mișcarea liniară a căruciorului, și un encoder de unghi, montat pe axul pendulului, care determină unghiul acestuia față de verticală. De asemenea, completarea stării cu restul componentelor presupune determinarea celor două viteze - liniară și unghiulară care nu se realizează direct și prin estimarea diferențelor numerice ale pozițiilor succesive date în timp real de senzori. Această operație amplifică zgomotul în semnalul dat de senzori, astfel se propune un filtru trece-jos discret care atenuează semnalele la înalte frecvențe pentru a obține o estimare suficient de precisă.

5.1.2 Implementarea pe microcontroler

Implementarea logicii de control asupra sistemului fizic se realizează pe microcontrolerul ESP32, ales pentru arhitectura sa dual-core care permite separarea logică a celor două task-uri critice sistemului. În primul rând, se urmărește generarea precisă a impulsurilor pentru motorul pas cu pas și pe de altă parte execuția algoritmului de control. ESP32 este un microcontroler cu nucleu dual având frecvență de lucru de până la 240 MHz și suficiente periferice pentru a interacționa direct cu encoderele optice dar și cu driverul motorului, fără nevoia de a utiliza resurse externe de calcul.

Primul nucleu este dedicat astfel exclusiv generării impulsurilor de comandă pentru motorul pas cu pas, rulând într-o buclă continuă care conferă o frecvență constantă impulsurilor, fără ca să fie afectate de întârzieri introduse de restul codului. Al doilea nucleu execută bucla principală de control, sincronizată la perioada de eșantionare dată anterior $T_s = 0.02$. De fiecare dată când senzorii detectează o mișcare se incrementează un contor, actualizat asincron prin întreruperi, din care se determină mai apoi poziția căruciorului și unghiul pendulului. Vitezele liniară și unghiulară sunt calculate prin diferențierea numerică a poziției, respectiv a unghiului, urmată de o filtrare trece-jos pentru reducerea zgomotului. Pe baza vectorului de stare $x_k = [x, \dot{x}, \theta, \dot{\theta}]^\top$, nucleul calculează comanda stabilizatoare sistemului și determină viteza de referință transmisă către primul nucleu responsabil de controlul motorului.

5.2 Implementarea rețelei neuronale pe microcontroler

5.2.1 Extragerea și calcularea legii de control ANN

Rețeaua neuronală $\mathcal{N}(8, 30)$, obținută în urma procedurii de pre-antrenare și re-antrenare descrise în Capitolul 3, este procesată inițial în mediul MATLAB sub formă de obiect **net**, ale cărui ponderi ($W^{(\ell)}$), bias-uri ($b^{(\ell)}$) și parametri de normalizare a intrării/ieșirii ($\mu_x, \sigma_x, \mu_u, \sigma_u$) sunt extrase din cod pentru a construi apoi legea de comandă în mediul *embedded*.

Rețeaua extrasă are forma dată de Ecuația 2.28, a cărei arhitectură este construită astfel încât să conțină ca strat de intrare $n_x = 4$ intrări (componentele vectorului de stare x_k), urmat de $L = 8$ straturi ascunse, fiecare cu $M = 30$ neuroni care sunt prelucrați prin activare ReLU și nu în ultimul rând un strat de ieșire liniar dat de comanda u_k .

Arhitectura rețelei este salvată într-un fișier extern de unde poate fi extrasă în prelucrările viitoare. În acest sens, odată ce avem toate datele cu privire la rețea putem construi legea de comandă în C pentru a controla pendulul prin ESP32 după Algoritmul 5.1:

Algoritm 5.1: Funcția care construiește Legea de comandă ANN

```

Input:  $x_k = (x_1, x_2, x_3, x_4) \in \mathbb{R}^4$ 
Output: Comandă  $u_k \in \mathbb{R}$ 
// Normalizare intrare
1 for  $i \leftarrow 1$  to  $n_x$  do
2    $l_i^{(0)} \leftarrow \frac{x_{k,i} - \mu_{x,i}}{\sigma_{x,i}};$ 
3 end
// Straturi ascunse cu activare ReLU
4 for  $\ell \leftarrow 1$  to  $L$  do
5   for  $i \leftarrow 1$  to  $M$  do
6      $\varphi \leftarrow b_i^{(\ell)} + \sum_{j=1}^{n_{\ell-1}} W_{ij}^{(\ell)} l_j^{(\ell-1)};$ 
7      $l_i^{(\ell)} \leftarrow \max(0, \varphi);$ 
8   end
9 end
// Strat de ieșire liniar
10  $z \leftarrow b_1^{(L+1)} + \sum_{j=1}^M W_{1j}^{(L+1)} l_j^{(L)};$ 
11 return  $\mu_u + \sigma_u \cdot z;$ 

```

5.2.2 Validarea legii de comandă ANN prin simulare

Pentru a verifica că rețeaua extrasă din mediul de simulare MATLAB oferă o aproximare corectă a legii de comandă, am realizat o simulare alternativă în Python în scopul verificării anticipate a rețelei înainte să fie testată în mediul fizic. În acest sens, rețeaua extrasă a fost simulată pe modelul construit în Capitolul 2 integrată numeric cu metoda Runge–Kutta (vezi 4.3). Totodată, pentru a testa robustețea controlerului au fost introduse în dinamica sistemului perturbații mărginite aditive $w_k \in W$. Cu toate aceste observații a fost construită legea de comandă u_k ale cărei efecte în simulare au fost ilustrate în Figura 5.1

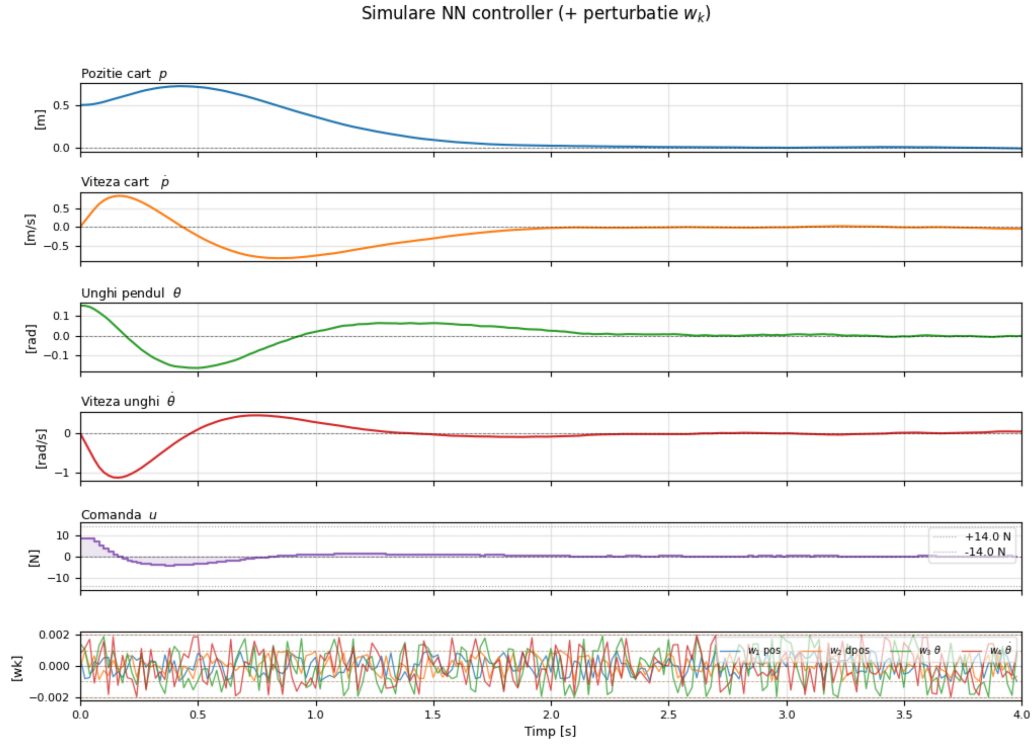


Figura 5.1: Simularea rețelei $\mathcal{N}(8, 30)$ în neliniar cu perturbăție aditivă w_k . Sunt ilustrate cele patru stări ale sistemului, comanda u_k aplicată și banda perturbăției aditivă.

Rezultatele simulării atestă faptul că sistemul se stabilizează în jurul originii, că rejectează perturbăția adăugată și că respectă constrângerile propuse.

5.3 Validarea legii de comandă în mediul fizic

În final, am realizat un ultim experiment pentru a valida metricile descrise anterior în simularea pe sistem fizic a legii de comandă generată prin construcția unei rețele neuronale ce aproximează o funcție afină pe regiuni dată de eMPC.

Figura 5.2 acoperă comportamentul pendulului la aplicarea legii de comandă NN pe mai multe intervale de simulare. Prima secțiune a graficului corespunde cu partea de homing/centering, urmată de mai multe etape unde comanda stabilizează pe eșantioane cu lungime diferită. Pendulul construit e unul de laborator, fiind realizat într-o formă simplificată, fără a urmări reproducerea fidelă a unui sistem industrial. Comanda este agresivă, din cauza necesității de a stabili rapid sistemul fizic neliniar, dar și pentru că legea de control propusă este doar o aproximare a unei probleme de optimizare cu constrângeri.

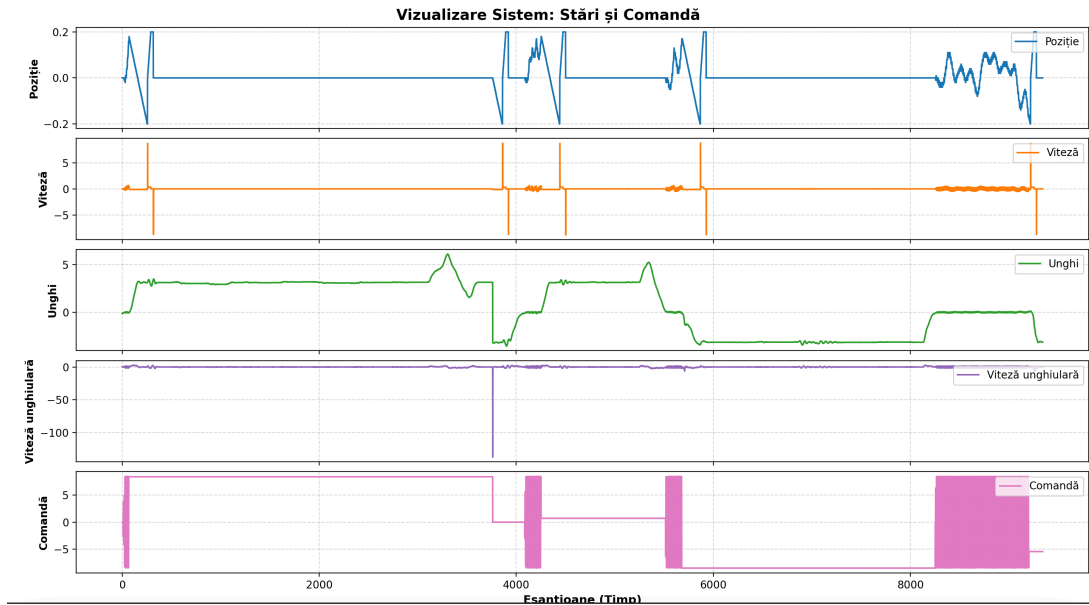


Figura 5.2: Răspunsul sistemului *pendul invers* controlat prin rețeaua neuronală care aproximează regulatorul eMPC.

În continuare, am ales o zonă de eșantionare în care comanda stabilizează sistemul, conferind în Fig 5.3 evoluția sistemului pe instalația fizică. Am observat cum valoarea unghiului oscilează ușor în jurul lui 0 (± 0.05 – 0.1 rad), viteza unghiulară oscilează constant între ± 1.5 rad/s pe toată durata, fără să fie atenuată, iar poziția căruciorului are o deplasare care oscilează lent (între -0.2 și $+0.11$ m).

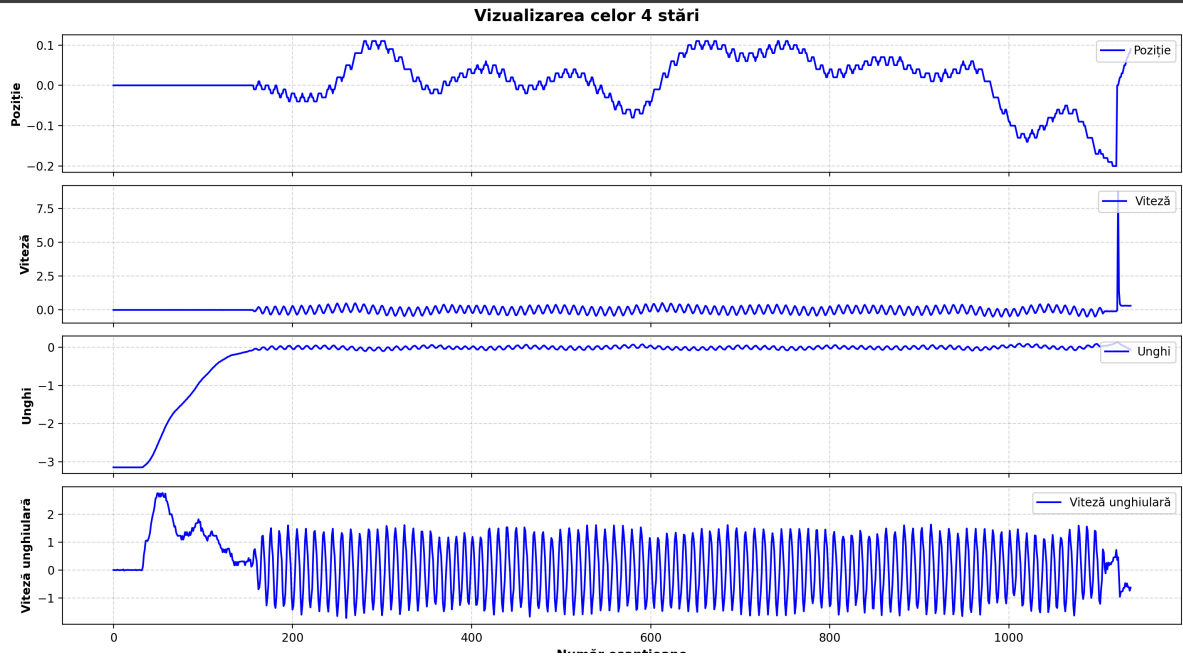


Figura 5.3: Evoluția stărilor pe pendul fizic într-un eșantion de simulare

Comanda u_{NN} are un caracter puternic oscilant, comutând la fiecare câțiva pași între ± 8 , din cauza faptului că legea de comandă nu reușește o stabilizare lină în jurul referinței.

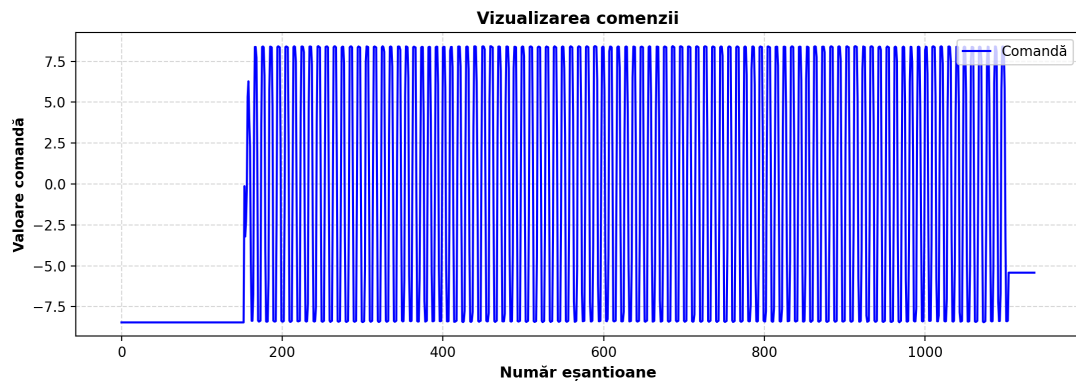


Figura 5.4: Comanda de control u_{NN} aplicată sistemului pendul-cărucior într-un eșantion de simulare.

6 Concluzii și Direcții Viitoare

6.1 Concluzii

În lucrarea de față am urmărit să proiectăm o strategie de control care să stabilizeze sistemul cu pendul invers prin generarea unei legi de comandă dată de o rețea neuronală care aproximează o lege de control afină date de eMPC cu garanții de robustețe prin metoda Tube MPC.

Inițial, am construit problema de (mp-QP) pentru modelul liniarizat al sistemului propus, cu scopul de a genera legea de comandă afină pe porțiuni. Pe baza acesteia am construit sursa de antrenament a rețelei și criteriul de evaluare a performanței rețelei. Apoi, am proiectat rețeaua neuronală $\mathcal{N}_{8,30}$ care aproximează legea afină dată de algoritmul eMPC urmată de o procedură de fine-tuning pentru reducerea erorii maxime de aproximare $\varepsilon_{\max}$.

S-a construit apoi tubul Z_{NN} cu ajutorul setului mRPI care atestă invarianța metodei la perturbații aditive. Tubul are integrat eroarea în cazul cel mai critic de aproximare a rețelei $\varepsilon_{\max}$, extinzând perturbația aditivă cu $|B| \cdot \varepsilon_{\max}$, realizându-se astfel garanții pentru fezabilitate recursivă, tubul asigurând respectarea constrângerilor pe stare și comandă prin $\tilde{X} = X \ominus Z$ și $\tilde{U} = U \ominus KZ$.

Metoda a fost testată inițial pe o stare inițială aleatoare, iar în simularea liniară am obținut o comandă u_{NN} care aproximează corect și conferă respectarea restricțiilor pe stare și intrare. Pentru a întări argumentul am realizat și o simulare statistică pe 1000 de stări inițiale din domeniul fezabil, iar rezultatele au confirmat rezultatul anterior. În ultimul rând, am realizat o simulare și în domeniul neliniar (integrare RK4), confirmând capacitatea rețelei de a asigura stabilitatea și în acest caz.

De asemenea, am testat și convergența spre origine prin funcția Lyapunov, iar aceasta a fost confirmată în aceste cazuri. Mai departe, am realizat comparația pe resurse (timp și memorie) și am văzut că metoda NN este net superioară celorlalte metode clasice de MPC, putând astfel fi implementată în sisteme de timp real.

În final, am realizat o ultimă simulare pe o instalație fizică de laborator, unde am verificat comportamentul sistemului la aplicarea comenzii u_{NN} dată de rețeaua neuronală care a reușit să stabilizeze sistemul propus în această lucrare.

6.2 Direcții Viitoare

O direcție viitoare ar consta în reducerea conservatorismului tubului Z_{NN} prin o strategie de antrenare mai bună orientată spre minimizarea $\varepsilon_{\max}$. Se poate construi o nouă funcție cost care ar introduce o penalizare specifică pentru erorile mari, fără să afecteze performanța aproximării de către rețea.

O altă direcție ar putea fi extinderea metodei pe sistemul complet neliniar și dezvoltarea algoritmilor necesari pentru dinamica neliniară cum ar fi NMPC. Așa cum am arătat anterior în Secțiunea 4.4, tubul Z nu mai păstrează invarianța pentru sistemul liniarizat, mai ales în

zonele pronunțate cu neliniaritate, în acest sens se pot utiliza tuburi neliniare sau alte tehnici de control neliniar.

O a treia direcție viitoare constă în determinarea unui set terminal invariant $\mathcal{X}_f^{NN}$ pentru rețeaua $\mathcal{N}_{8,30}$ care ar asigura formal stabilitatea asimptotică a sistemului. În lucrarea de față, în secțiunea 4.2 a fost aproximat acest set terminal cu matricea P dată de ecuația Riccati a formulării MPC.

Bibliografie

- [1] J.M. Maciejowski. *Predictive Control with Constraints*. Harlow, UK: Prentice Hall, 2002.
- [2] Ș.S. Mihai, F. Stoican și B.D. Ciubotaru. „Computing the Explicit MPC Solution Using the Hasse Diagram of the Lifted Feasible Domain“. În: *Proceedings of the 2023 European Control Conference (ECC)*. Bucharest, Romania: IEEE, 2023, pag. 1–7. DOI: [10.23919/ECC57647.2023.10178222](https://doi.org/10.23919/ECC57647.2023.10178222).
- [3] M. Schulze Darup. „Exact Representation of Piecewise Affine Functions via Neural Networks“. În: *Proceedings of the 2020 European Control Conference (ECC)*. Saint Petersburg, Russia: IEEE, 2020, pag. 1073–1078. DOI: [10.23919/ECC51009.2020.9143983](https://doi.org/10.23919/ECC51009.2020.9143983).
- [4] D.Q. Mayne, M.M. Seron și S.V. Raković. „Robust model predictive control of constrained linear systems with bounded disturbances“. În: *Automatica* 41.2 (2005), pag. 219–224. DOI: [10.1016/j.automatica.2004.08.019](https://doi.org/10.1016/j.automatica.2004.08.019).
- [5] Sorin Olaru. „Predictive Control“. Lecture notes, Master ATSI, CentraleSupélec. 2025.
- [6] C.D. Green. *Equations of Motion for the Cart and Pole Control Task*. <https://sharpneat.sourceforge.io/research/cart-pole/cart-pole-equations.html>. 2024.
- [7] Andrei Vener. „Synthesis of a Predictive Control Law for a Nonlinear System“. Bachelor’s thesis. Bucharest, Romania: Faculty of Automatic Control și Computers, University Politehnica of Bucharest, iul. 2024.
- [8] Stephen Boyd. „Convex optimization of graph Laplacian eigenvalues“. en. În: mai 2007, pag. 1311–1319. DOI: [10.4171/022-3/63](https://doi.org/10.4171/022-3/63). URL: <https://ems.press/doi/10.4171/022-3/63> (vizitat 12/03/2024).
- [9] B. Karg și S. Lucia. „Efficient Representation and Approximation of Model Predictive Control Laws via Deep Learning“. În: *IEEE Transactions on Cybernetics* 50.9 (2020), pag. 3866–3878. DOI: [10.1109/TCYB.2020.2999556](https://doi.org/10.1109/TCYB.2020.2999556).
- [10] S.V. Raković și alții. „Invariant Approximations of the Minimal Robust Positively Invariant Set“. În: *IEEE Transactions on Automatic Control* 50.3 (2005), pag. 406–410. DOI: [10.1109/TAC.2005.843854](https://doi.org/10.1109/TAC.2005.843854).
- [11] A. Girard. „Reachability of Uncertain Linear Systems Using Zonotopes“. În: *Hybrid Systems: Computation and Control (HSCC 2005)*. Vol. 3414. Lecture Notes in Computer Science. Zurich, Switzerland: Springer, 2005, pag. 291–305. DOI: [10.1007/978-3-540-31954-2_19](https://doi.org/10.1007/978-3-540-31954-2_19).
- [12] F. Stoican, C. Oară și M. Hovd. „RPI Approximations of the mRPI Set Characterizing Linear Dynamics with Zonotopic Disturbances“. În: *Developments in Model-Based Optimization and Control*. Ed. de S. Olaru, A. Grancharova și F. Lobo Pereira. Vol. 464. Lecture Notes in Control and Information Sciences. Springer, 2015, pag. 329–348. DOI: [10.1007/978-3-319-26687-9_17](https://doi.org/10.1007/978-3-319-26687-9_17).
- [13] Diederik P Kingma și Jimmy Ba. „Adam: A method for stochastic optimization“. În: *arXiv preprint arXiv:1412.6980* (2014).
- [14] Franco Blanchini și Stefano Miani. *Set-Theoretic Methods in Control*. Systems & Control: Foundations & Applications. Cham: Springer International Publishing, 2015. ISBN: 978-3-319-17932-2. DOI: [10.1007/978-3-319-17933-9](https://doi.org/10.1007/978-3-319-17933-9). URL: <https://link.springer.com/10.1007/978-3-319-17933-9>.

- [15] Joel A. E. Andersson și alții. „CasADi – A software framework for nonlinear optimization and optimal control“. În: *Mathematical Programming Computation* 11.1 (2019), pag. 1–36. DOI: [10.1007/s12532-018-0139-4](https://doi.org/10.1007/s12532-018-0139-4).

Contribuții personale

Reprezentare Eficientă pentru Explicit MPC folosind Rețele Neuronale Artificiale

Raul-Adrian Lungulescu

Daniel Ioan, Florin Stoican

	Activitate	Durată [Zile]
1	Studiul teoriei MPC și Tube-MPC	14
2	Modelarea și liniarizarea pendulului invers	2
3	Implementarea eMPC	5
4	Calculul setului mRPI prin aproximare zonotopică (Raković)	10
5	Antrenarea rețelei neuronale $\mathcal{N}_{8,30}$ și procedura de fine-tuning	14
6	Construcția tubului robust $\mathcal{Z}_{NN}$ cu garanții de invarianță	7
7	Simulări numerice: sistem liniar, neliniar RK4, validare Monte Carlo	10
8	Implementarea rețelei neuronale pe microcontrolerul ESP32	5
9	Validarea experimentală pe sistemul fizic	5
10	Redactarea lucrării de licență	15
Total		87